《软件工程》课程贯穿项目

CampusOverflow 校园问答平台

软件项目开发实验指导书

模块化单体 × 领域驱动设计 × Scrum 敏捷开发 × GitHub 团队协作

适用对象：计算机类专业本科生（4 ~ 5 人/组）

开发周期：8 周（1 个启动 Sprint + 3 个开发 Sprint + 1 个发布 Sprint）

技术栈：Java 21 · Spring Boot 3.4 · MySQL 8 · Redis 7 · React 18 · GitHub Actions

开发工具：IntelliJ IDEA · Git / GitHub · Docker

版本 1.0 ｜ 2026 年 10 月

目 录

第 1 章 实验概述 3
1.1 实验目标 3
1.2 项目简介 3
1.3 实验约束与总体安排 4
1.4 起始代码现状 5
1.5 与起始代码 README 中原计划的差异 5
第 2 章 系统架构解析 7
2.1 技术栈 7
2.2 模块化单体与六边形分层（ADR-001） 7
2.3 架构适应度函数（Fitness Functions） 8
2.4 架构决策记录（ADR）摘要 9
2.5 关键横切机制 9
2.6 运行时视图：场景 R-2“采纳最佳答案” 11
第 3 章 领域驱动设计（DDD）分析 12
3.1 子域划分 12
3.2 上下文映射（Context Map） 12
3.3 统一语言（Ubiquitous Language）词汇表 13
3.4 各上下文的领域模型 14
3.5 领域事件目录 16
3.6 战术设计编码约定 17
3.7 错误码约定 18
第 4 章 团队组织与 Scrum 框架 20
4.1 Scrum 角色 20
4.2 Scrum 事件与时间盒 20
4.3 Scrum 工件及其在 GitHub 上的载体 21
4.4 就绪定义（DoR）与完成定义（DoD） 21
4.5 估算与容量 22
第 5 章 Product Backlog 23
5.1 用户角色 23
5.2 史诗（Epics） 23
5.3 排序原则 23
5.4 完整 Product Backlog（已排序） 24
5.5 依赖关系 27
第 6 章 Sprint 规划与执行指南 28
6.1 Sprint 总览 28
6.2 Sprint 0（第 1 周）： 启动与基础设施 28
6.3 Sprint 1（第 2–3 周）： 问答核心 30

6.4 Sprint 2（第 4–5 周）：声誉与激励 34
6.5 Sprint 3（第 6–7 周）：内容发现与端到端贯通 37
6.6 Sprint 4（第 8 周）：稳定化与发布 39
第 7 章 Git / GitHub 团队协作规范 41
7.1 协作模式：PO 个人仓库 + 协作者 41
7.2 分支策略 43
7.3 提交信息规范（Conventional Commits） 44
7.4 Issue 与 Projects 看板 44
7.5 Pull Request 与 Code Review 45
7.6 IntelliJ IDEA 配置要点 46
第 8 章 CI/CD 与质量门禁 48
8.1 流水线结构 48
8.2 为什么需要“渐进式质量门禁” 48
8.3 各 Sprint 的质量门槛 49
8.4 持续交付 49
8.5 CI 红灯处理流程 50
第 9 章 Sprint 验收与考核评价 51
9.1 Sprint 评审会的验收内容 51
9.2 评分方案 51
9.3 学术诚信与红线 52
9.4 最终交付物清单 52
附录 54
附录 A 107 个 TODO 方法与 PBI / Sprint 映射 54
附录 B 模板 55
附录 C 常见问题与排错 56
附录 D REST 接口清单（摘自接口层代码） 57

（若目录未显示，请在 Word 中右键目录区域选择“更新域”→“更新整个目录”。）

第 1 章 实验概述

1.1 实验目标

本实验以一个“只完成了一部分”的真实规模项目——CampusOverflow（大学校园版 Stack Overflow）——为载体。各小组在 8 周内以 Scrum 方式迭代交付，最终得到一个可部署、可演示、通过全部自动化质量门禁的系统。完成实验后，学生应能够：

维度	具体目标
知识	理解模块化单体、六边形分层、限界上下文、聚合、领域事件、事务性发件箱、最终一致性等概念，并能在代码中指出其落点。
能力（工程）	能阅读并扩展一个 27 个 Maven 模块的工程；用测试驱动（红→绿→重构）实现领域规则；编写不依赖容器的应用层单元测试；让 ArchUnit 架构测试、覆盖率门禁与端到端集成测试保持全绿。
能力（协作）	熟练使用 Git 分支、Pull Request、Code Review、Issue、Projects 看板与 GitHub Actions；能在团队内拆分任务、处理合并冲突、修复 CI 红灯。
能力（过程）	能编写用户故事与验收标准、估算故事点、制定 Sprint Goal，并主持 Sprint 计划会、每日站会、评审会与回顾会。
素养	遵守“不改测试断言、不删测试”的学术诚信约定；以可运行软件和可追溯的提交记录证明自己的贡献。

1.2 项目简介

CampusOverflow 是面向高校师生的课程问答社区。学生可以按课程提问、回答、投票与评论；提问者或课程教师/助教可以采纳最佳答案；教师可以为优质回答打上“教师认证”标记，也可以发起积分悬赏。系统用**声誉**激励回答者，用**徽章**和**排行榜**营造学习氛围，并提供中文全文检索、相关问题推荐和站内通知。

系统被划分为四个限界上下文（Bounded Context）：

上下文	职责	起始代码状态
Identity 身份与课程	注册/登录、校园身份认证、教职工账号、角色、课程与课程成员	**完整参考实现**（读懂、模仿，可扩展）
Q&A 问答	问题、回答、评论、投票、采纳、教师认证、软删除、问题列表与详情	骨架就绪，**36 个方法待实现**

上下文	职责	起始代码状态
Reputation 声誉与激励	声誉账本、投票/采纳计分、每日上限、徽章、悬赏、排行榜、对账	骨架就绪，**39 个方法待实现**
Discovery 内容发现	搜索投影、中文全文检索、相关问题推荐、@提及与站内通知	骨架就绪，**32 个方法待实现**

1.3 实验约束与总体安排

约束项	要求
软件架构	模块化单体（Modular Monolith）+ 六边形分层；跨上下文只经 `*-api` 发布语言或集成事件通信（见 ADR-001）。
设计方法	领域驱动设计（DDD）：战略设计（子域、限界上下文、上下文映射、统一语言）+ 战术设计（聚合、值对象、领域服务、仓储、领域事件）。
团队	4～5 人一组，全员参与领域层编码；每个上下文设一名“主责人”，但不搞“只写前端/只写文档”的分工。
协作平台	Git + GitHub：由团队 PO 在本地 IntelliJ 导入起始代码并推送到 **PO 个人 GitHub 仓库**，其他成员以协作者身份从该仓库克隆并推送功能分支（不设仓库管理员）；使用受保护分支、Issue、Projects 看板、Pull Request、Code Review、GitHub Actions。
开发工具	IntelliJ IDEA（Ultimate 教育版或 Community）+ JDK 21 + Maven 3.9 + Docker Desktop + Node.js 20。
周期	8 周，共 5 个 Sprint：Sprint 0（第 1 周）→ Sprint 1（第 2–3 周）→ Sprint 2（第 4–5 周）→ Sprint 3（第 6–7 周）→ Sprint 4（第 8 周）。
过程	Scrum：Product Backlog 按优先级排序；每个 Sprint 聚焦一个 Sprint Goal；以“可运行的软件 + GitHub 协作证据 + CI 状态”验收。

8 周路线图（每个 Sprint 结束都必须得到一个可运行、可演示的增量，并在 `main` 分支打版本标签）：

Sprint	周次	Sprint Goal（一句话）	版本标签
Sprint 0 启动	第 1 周	团队、仓库、环境、CI 与 Backlog 全部就绪，能注册/登录并加入课程	v0.1.0
Sprint 1 问答核心	第 2–3 周	学生可以完成“提问→回答→投票→评论→采纳”的完整问答闭环	v0.2.0
Sprint 2 声誉激励	第 4–5 周	问答行为能异步、幂等地转化为声誉、徽章、悬赏与排行榜	v0.3.0
Sprint 3 内容发现	第 6–7 周	用户能搜到问题、看到相关推荐并收到通知，端到端链路全绿	v0.9.0（RC）
Sprint 4 发布	第 8 周	系统稳定化、体验打磨、可观测，完成 v1.0 发布与最终答辩	v1.0.0

1.4 起始代码现状

类别	已提供（读懂它，不要重写）	需要实现 / 补充
工程骨架	父 POM + 27 个 Maven 模块（共享内核 1 + 四个上下文×5 层 + bootstrap 1 + 聚合 POM）	—
共享内核	领域异常体系（422/409/403/404）、Guard、Actor/Role、IntegrationEvent、发件箱端口、幂等端口、审计端口、分页	—
Identity	领域、应用、基础设施、接口四层完整实现及测试	可选扩展（禁用账号、个人资料、选课审批等）
Q&A / Reputation / Discovery	`*-api`（发布语言与事件）、仓储接口、JPA 实体与适配器、Redis/JDBC 适配器、REST 控制器、Flyway 迁移	领域层聚合与规则、应用层用例编排：共 **107 个 TODO(Sx) 方法 / 32 个文件**
平台能力	安全链（Redis 会话 + CSRF）、发件箱中继、全局异常处理（RFC 9457）、OpenAPI、演示数据、Docker、CI 流水线	按 Sprint 渐进开启质量门禁（见 8.3 节）
测试	领域测试（Q&A/Reputation/Discovery）、Q&A 应用层用例测试与手写 Fake、ArchUnit 架构测试、端到端集成测试（Testcontainers）	Reputation、Discovery **应用层单元测试需自行编写**；补充边界测试使领域覆盖率 ≥ 70%
前端	React 骨架：登录注册、问题列表与搜索、详情（投票/采纳/认证）、提问、排行榜、通知	课程页、编辑/删除、悬赏对话框、声誉主页、搜索筛选、通知已读、体验优化

> **提示｜为什么一上来测试是红的**
>
> 克隆后运行 `mvn test` 会看到约 38 个失败的测试——这不是环境问题，而是本学期的“需求说明书”。**测试即规格**：先读测试，再写实现。
>
> 允许新增方法、类和测试；**禁止修改已有测试的断言或删除测试**（相当于篡改需求）。确信测试与文档矛盾时，在 PR 中说明并与教师确认。

1.5 与起始代码 README 中原计划的差异

起始代码 README 第 4 节按 15 周（S0–S4）规划。本指导书根据“8 周完成”的约束重新排期，主要调整为：

- 将“读懂 Identity + 架构文档”压缩为 1 周的 Sprint 0，并把团队协作基础设施（分支保护、看板、模板、CI 门禁）前置到 Sprint 0 完成。

- 原 S3（Reputation + Discovery，71 个方法）拆成两个 2 周 Sprint：Sprint 2 只做 Reputation，Sprint 3 做 Discovery + 端到端贯通，避免单个 Sprint 过载。
- 原 S1“扩展 Identity”调整为低优先级 Backlog 项，放在 Sprint 4 按剩余容量选做（Product Owner 依价值排序后决定）。
- 引入“行走骨架（Walking Skeleton）”与“渐进式质量门禁”两项工程实践，保证每个 Sprint 结束时 CI 是绿的、软件是可运行的（详见 6.2 与 8.3 节）。

第 2 章 系统架构解析

在 Sprint 0 中，每位成员都要能对照代码讲清楚本章的每一张表。

2.1 技术栈

层次	技术与版本	在本项目中的作用
语言/运行时	Java 21（虚拟线程可选开关）	`record` 值对象、`switch` 模式匹配、不可变集合
应用框架	Spring Boot 3.4.5（Web、Data JPA、Security、Session、Actuator）	只出现在 application / infrastructure / interfaces / bootstrap 层
数据库	MySQL 8.0.39（utf8mb4、`ngram_token_size=2`、关闭停用词、UTC）	业务表按上下文前缀隔离；ngram 全文索引支持中文检索
迁移	Flyway（每个上下文独立迁移目录）	表结构只由迁移脚本管理，`ddl-auto: none`
缓存/会话	Redis 7 + Spring Session	服务端会话、登录限流、问题列表缓存、浏览计数、排行榜 ZSET
接口契约	springdoc-openapi 2.8.6；RFC 9457 Problem Details	Swagger UI：`/swagger-ui.html`；错误响应带业务 `code`
内容安全	commonmark 0.24 + OWASP Java HTML Sanitizer	Markdown 渲染与 XSS 白名单净化（质量场景 QS-07）
测试	JUnit 5、AssertJ、ArchUnit 1.3.0、JaCoCo 0.8.12、Testcontainers、Awaitility	领域测试、用例测试、架构适应度函数、覆盖率门禁、端到端测试
前端	React 18 + React Router 6 + TypeScript 5 + Vite 5	SPA，开发端口 5173，经 Vite 代理访问后端
交付	Docker / Docker Compose、Nginx、Prometheus、Grafana、GitHub Actions	一键部署、指标监控、CI 五阶段流水线 + 镜像构建

2.2 模块化单体与六边形分层（ADR-001）

整个后端是**一个可部署单元**，内部划分四个限界上下文；每个上下文再分为 5 个 Maven 模块（5 层）。Maven 依赖图 + ArchUnit 测试共同“硬性”守住边界。

代码清单 2-1 后端模块结构

```
backend/
├── co-shared-kernel/            共享内核：异常、Guard、Actor、事件契约、分页（零框架依赖）
├── co-identity/                 Identity 上下文（完整参考实现）
│   ├── co-identity-api/          发布语言：IdentityApi、UserSummary、UserRegisteredEvent
│   ├── co-identity-domain/       领域层：User、Course、CourseMember、PasswordPolicy、仓储接口
│   ├── co-identity-application/  应用层：注册、课程、管理员用例（@Transactional 边界）
│   ├── co-identity-infrastructure/ 基础设施：JPA 实体与适配器、BCrypt、字段加密、Redis 限流
│   └── co-identity-interfaces/   接口层：AuthController、CourseController、AdminUserController
├── co-qa/           （同上 5 层）Q&A 上下文
├── co-reputation/   （同上 5 层）Reputation 上下文
├── co-discovery/    （同上 5 层）Discovery 上下文
└── co-bootstrap/    启动装配：安全链、发件箱、全局异常、OpenAPI、演示数据、ArchUnit 与集成测试
```

层	职责	允许依赖	典型类
api	发布语言（对外契约）：查询接口、只读 DTO、集成事件 record	仅 JDK 与共享内核	`QaQueryApi`、`QuestionRef`、`AnswerAcceptedEvent`
domain	业务规则与不变量：聚合根、实体、值对象、领域策略、仓储接口	共享内核（**禁止** Spring/JPA/Jackson）	`Question`、`Vote`、`Bounty`、`ReputationRulebook`
application	用例编排：加载聚合→调用领域方法→保存→发布事件→失效缓存→审计；定义事务边界与端口	domain、本上下文及他上下文的 api	`AnswerCommandService`、`BountyService`
infrastructure	端口实现：JPA 实体与映射、JDBC 读模型、Redis、渲染器、定时任务	domain、application	`QuestionRepositoryAdapter`、`RedisLeaderboardStore`
interfaces	REST 控制器、请求校验、把安全主体转换为 `Actor`	application	`QuestionController`、`BountyController`

2.3 架构适应度函数（Fitness Functions）

`co-bootstrap/src/test/java/.../architecture/ArchitectureTest.java` 把架构约束写成自动化测试，CI 第 3 阶段单独运行：

编号	规则	违反时的典型错误
FF-1	`..domain..` 包不得依赖 Spring、JPA、Validation、Hibernate、Jackson、Swagger	在聚合上加 `@Entity`、在领域层用 `@Component`
FF-2	一个上下文只能依赖其他上下文的 `api` 包	Reputation 直接 import `com.campusoverflow.qa.domain.Question`
FF-3	上下文之间无循环依赖	Q&A 反过来依赖 Reputation 的 api

编号	规则	违反时的典型错误
FF-4	六边形分层方向：依赖只能由外向内	领域层调用应用服务；控制器直接调用仓储适配器
FF-5	`api` 包只依赖 JDK 与共享内核	事件 record 里引用领域类型或 Spring 类型
FF-6	JPA 实体位于 infrastructure，且表名使用本上下文前缀（`id_`/`qa_`/`rep_`/`dis_`）	新建表忘记前缀；实体放进 domain 包
FF-8	声誉对账：账户余额 = 流水之和（定时任务 + 指标 `co_reputation_mismatches`）	直接 `UPDATE rep_account` 而不写流水

2.4 架构决策记录（ADR）摘要

ADR	决策	对实现的直接影响
001	模块化单体 + 六边形分层	按上下文/层组织代码；跨上下文只用 api 或事件
002	跨上下文集成使用事务性发件箱	应用服务中 `events.publish(...)` 必须在 `@Transactional` 内；消费者必须幂等
003	Redis 服务端会话而非 JWT	前端用 Cookie + `X-XSRF-TOKEN`；封禁可通过删除会话即时生效
004	声誉只追加账本 + 定期对账	任何声誉变化都先写 `rep_ledger`，唯一键 `(event_id, user_id, reason)` 保证幂等
005	MySQL ngram 全文索引	关键词每个词项 ≥ 2 字符（DIS-1001）；搜索封装在 `SearchPort` 后，可切换 LIKE 实现
006	用户内容一律软删除	`deleted_at` 标记；已删除内容不出现在列表/详情/搜索；删除写审计
007	前后端以 OpenAPI 契约协作	接口变更先改契约；分页统一 `{items,total,page,size}`；时间统一 UTC ISO-8601
008	领域对象与 JPA 实体分离	聚合不回写计数器（score/answerCount/viewCount），由仓储原子 `UPDATE` 维护

2.5 关键横切机制

2.5.1 事务性发件箱与至少一次投递

1. 应用服务在同一本地事务中：保存聚合 + 调用 `IntegrationEventPublisher.publish(event)`，后者把事件 JSON 写入 `outbox_event` 表。
2. `OutboxRelay` 每 500 ms 用 `SELECT … FOR UPDATE SKIP LOCKED` + 租约认领未投递事件，按 id 顺序通过 Spring 进程内事件总线分发给所有 `@EventListener`。

3. 任一监听器抛异常 → 本条事件记为失败、指数退避重试，**并停止本批后续事件**（保序）；超过 10 次成为死信（指标 `co_outbox_dead_letters`）。
4. 因为是"至少一次"，消费者必须幂等：Reputation 用账本唯一键；Discovery 投影用 `ProcessedEventStore.markProcessed(eventId, consumer)`；通知用 `(source_event_id, recipient_id)` 唯一键。

> **注意｜对增量开发的影响（务必阅读）**
>
> 第 3 条意味着：只要还有一个上下文的监听器在抛 `UnsupportedOperationException: TODO`，所有事件都会投递失败并"堵住"发件箱。
>
> 因此 Sprint 1 开始时要实施"行走骨架"：把尚未进入当前 Sprint 的监听器方法**暂时改为空实现**（只打 DEBUG 日志），并为其建立 `tech-debt` Issue；到对应 Sprint 再实现（见 6.2.3）。

2.5.2 安全与身份

- 会话 Cookie `SESSION`（HttpOnly、SameSite=Lax）；CSRF 双提交：先 `GET /api/v1/auth/csrf` 取得 `XSRF-TOKEN` Cookie，写请求带 `X-XSRF-TOKEN` 头。
- 接口层通过 `ActorArgumentResolver` 把登录主体解析为 `Actor(userId, displayName, role)` 显式传给应用服务——应用层与领域层因此不依赖 Spring Security，单元测试可任意构造身份。
- "课程管理者"判定统一走 `IdentityApi.canManageCourse(userId, courseId)`：管理员；或该课程教师/助教；未关联课程时全局教师亦可。领域方法用参数 `actorManagesCourse` 接收该结果。

2.5.3 错误处理

领域层只抛共享内核中的四类异常，`GlobalExceptionHandler` 统一映射为 RFC 9457 Problem Details 并带业务 `code`：`BusinessRuleException`→422、`ConflictException`→409、`ForbiddenException`→403、`NotFoundException`→404；乐观锁失败→409 `SYS-409-CONCURRENT`；唯一约束冲突→409 `SYS-409-DUPLICATE`。完整错误码约定见 3.7 节。

2.5.4 读写分离（CQRS-lite）与缓存

- 写路径：命令服务操作聚合；计数器由仓储 `adjustScore / adjustAnswerCount` 原子更新。
- 读路径：`QaReadModel` 直接返回投影行（`QuestionRow/AnswerRow/CommentRow`），用 `IdentityApi.findUsers` 批量取作者，避免 N+1。
- 问题列表 Cache-Aside（TTL 60 s + 抖动），任何写操作后 `listCache.invalidateAll()`；浏览数先写 Redis 再定时批量落库。

2.6 运行时视图：场景 R-2“采纳最佳答案”

这是贯穿 Sprint 1 ~ 3 的核心场景，也是端到端集成测试 `CampusOverflowFlowIT` 验证的链路：

1. 提问者调用 `PUT /api/v1/questions/{id}/accepted-answer` → `AnswerCommandService.accept`。
2. 加载 `Question` 与 `Answer`，查询 `canManageCourse`，调用 `Question.accept(answer, actor, manages)` 守护不变量（只能采纳一次、答案属于本题、未删除、有权限）。
3. 同一事务内：保存问题 + 发布 `AnswerAcceptedEvent` 到发件箱 + 写审计 `ANSWER_ACCEPTED` + 失效列表缓存 → 返回 204。
4. 中继投递事件：Reputation 给回答者记 +15（自问自答 0 分）、授予徽章、结算悬赏 → 发布 `ReputationChangedEvent / BadgeAwardedEvent / BountyAwardedEvent`。
5. Discovery：搜索投影标记“已采纳”；通知中心给回答者推送“你的回答被采纳”；排行榜监听 `ReputationChangedEvent` 更新 Redis ZSET。
6. 前端稍后刷新即可看到声誉 +15、徽章“初露锋芒”、搜索结果“已采纳”与通知——**最终一致**，演示时需向评委解释数百毫秒的延迟。

第 3 章 领域驱动设计（DDD）分析

本章通过对起始代码的逆向分析，还原出项目的 DDD 战略设计与战术设计。Sprint 0 的“事件风暴”工作坊应以本章为参考答案进行对照与补充，产出物提交到仓库 `docs/ddd/` 目录。

3.1 子域划分

子域	类型	对应上下文	判断依据
课程问答	核心域	Q&A	平台存在的理由；“只能采纳一次”“已删除不可回答”等规则最复杂、最体现差异化
激励与声誉	核心域	Reputation	决定社区能否持续活跃；账本、悬赏、反作弊规则是本校定制的“游戏规则”
身份与课程	支撑子域	Identity	必要但不独特；校园身份认证、课程成员关系是授权依据
内容发现	通用/支撑子域	Discovery	搜索、推荐、通知是通用能力，可替换为 Elasticsearch 或独立服务（ADR-001 演进路径中的第一拆分候选）

3.2 上下文映射（Context Map）

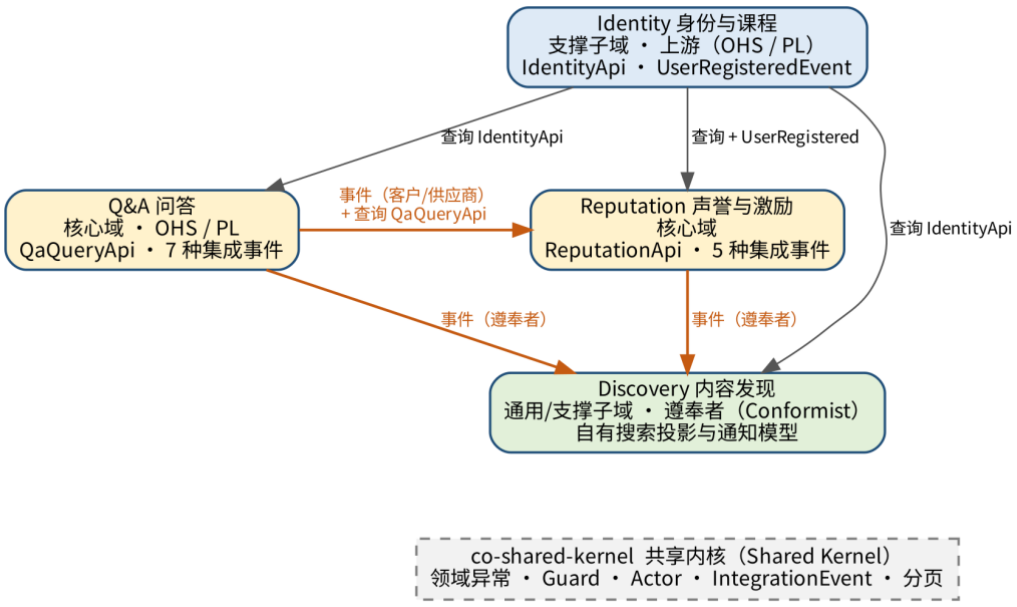


图 3-1 上下文映射图（箭头指向下游；橙色为异步集成事件）

上游 → 下游	关系模式	集成方式	说明
Identity → Q&A / Reputation / Discovery	开放主机服务 + 发布语言（OHS/PL）	同步查询 `IdentityApi`；异步 `UserRegisteredEvent`	下游只拿到 `UserSummary`（不含学号、邮箱等个人信息）
Q&A → Reputation	客户/供应商（Customer/Supplier）	异步：Answer*/VoteCast 事件；同步：`QaQueryApi.findQuestion`	Reputation 需要的事件字段（如 `questionAuthorId`）由 Q&A 在事件中提供
Q&A → Discovery	遵奉者（Conformist）	异步：Question*/Answer*/Vote/Comment/ContentDeleted 事件	Discovery 把事件折叠为自己的 `SearchDocument` 投影
Reputation → Discovery	遵奉者（Conformist）	异步：Bounty*/BadgeAwarded 事件	用于“悬赏中”筛选与悬赏/徽章通知
全部上下文	共享内核（Shared Kernel）	Maven 依赖 `co-shared-kernel`	修改共享内核须全组评审（影响所有上下文）

3.3 统一语言（Ubiquitous Language）词汇表

代码中的类名、方法名、错误信息、Issue 标题、站会发言都要使用同一套词汇。下表为从代码中提取的核心术语：

术语	代码名	定义 / 规则
问题	`Question`	学生发布的求助；标题 10–150 字符、正文 Markdown、1–5 个标签，可关联课程
回答	`Answer`	对问题的解答；独立聚合，经 `questionId` 引用问题
采纳	`accept`	提问者或课程管理者选定最佳答案；每题最多一次且不可更改
教师认证	`endorse`	课程教师/助教为优质回答打的标记（`endorsedBy`），每个回答只能认证一次
投票	`Vote`	对问题/回答 +1 或 −1；每人每内容一票；改投、撤销（值为 0）都可以
评论 / 回复	`Comment`	挂在问题或回答下，最多一层回复，回复必须与父评论在同一目标上
标签	`TagName` / `Tags`	统一小写、去掉前导 #，支持中文与 `# + . - _`；每题去重后 1–5 个
软删除	`softDelete` / `deletedAt`	标记删除而非物理删除；已删除内容不可再回答/编辑/采纳
课程管理者	`actorManagesCourse`	管理员，或该课程教师/助教（由 `IdentityApi.canManageCourse` 判定）

术语	代码名	定义 / 规则
声誉	`reputation`	社区信誉分，只能由账本流水改变；采纳 +15、被赞 +10、被踩 −2
账本流水	`LedgerEntry`	只追加的声誉变动记录；`(eventId, userId, reason)` 唯一
每日上限	`DAILY_VOTE_CAP`	每人每个 UTC 自然日因被投票获得的正向声誉最多 200
悬赏	`Bounty`	课程管理者用悬赏积分为问题加码；OPEN → AWARDED / EXPIRED
悬赏积分	`availablePoints` / `frozenPoints`	与声誉不同的“货币”；发起悬赏时冻结，结算时转出或退回
徽章	`BadgeType`	初来乍到、初露锋芒、解惑达人、名师认可、小有名气、德高望重
排行榜周期	`LeaderboardPeriod`	WEEK / MONTH / TERM / ALL；学期：2–7 月春季，8 月–次年 1 月秋季
对账	`ReconciliationService`	检查每个账户的声誉是否等于其流水之和（FF-8）
搜索投影	`SearchDocument` / `dis_search_doc`	Discovery 自己的问题模型，只关心“如何被找到”
相关问题	`RelatedQuestionPolicy`	按标签集合 Jaccard 相似度推荐；平局时得分高、已采纳者优先
提及	`MentionParser`	评论中的 `@昵称`，去重，每条最多提醒 5 人
通知	`Notification`	站内消息；`(sourceEventId, recipientId)` 唯一，只有接收者能标记已读
集成事件	`IntegrationEvent`	跨上下文传播的不可变 record，带全局唯一 `eventId`
发布语言	`*-api` 模块	上下文对外暴露的稳定契约（查询接口 + DTO + 事件）
操作者	`Actor`	当前操作主体（userId、displayName、role），由接口层显式传入

3.4 各上下文的领域模型

3.4.1 Identity（参考实现，供模仿）

构造块	类型	要点
`User`	聚合根	工厂方法 `register`（学生）/ `createStaff`（教职工，须管理员）；`verify`、`changeRole`、`disable` 均先校验管理员身份；`reconstitute` 供持久化重建，不做校验
`Course` / `CourseMember`	聚合根 / 实体	只有教师或管理员能建课；课程教师的课程内角色不可被降级
`PasswordPolicy`	领域策略（静态）	8–64 位、同时含字母与数字（ID-1004）
`UserRepository` / `CourseRepository`	仓储接口	接口在领域层、实现在基础设施层（依赖倒置）
`UserRegistrationService`	应用服务	经典编排模板：校验→工厂→查重→保存→发布事件→审计（务必精读）

3.4.2 Q&A 问答上下文

构造块	类型	状态 / 字段	待实现方法
`Question`	聚合根	title、body、tags、acceptedAnswerId、deletedAt；计数器只读	`post edit accept softDelete ensureOpenForAnswers ensureNotDeleted`
`Answer`	聚合根	questionId、body、endorsedBy、deletedAt（是否采纳由 Question 决定）	`submit edit endorse softDelete ensureNotDeleted`
`Vote`	聚合	voterId、targetType、targetId、value∈{−1,+1}	`cast change checkValue`
`Comment`	实体（独立聚合）	targetType、targetId、questionId、parentId、body	`post`
`Title` / `MarkdownBody` / `Tags` / `TagName`	值对象（record）	在构造时完成校验与规范化	`MarkdownBody.ofQuestion/ofAnswer`、`Tags.names`
4 个仓储接口 + `TagRepository`	仓储接口	计数器用 `adjustScore/adjustAnswerCount/adjustUsage` 原子维护	—（已由基础设施实现）

Q&A 核心不变量（来自 `Question` 类注释与领域测试）：

- 一个问题最多一个被采纳答案，采纳后不可更改（重复采纳 → 409 `QA-2002`）。
- 只有提问者或课程管理者可以采纳（否则 403 `QA-3001`）；被采纳的回答必须属于本题且未删除（否则 422）。
- 已删除的问题不能再被回答、编辑或采纳；提问者只能删除尚无回答的问题，课程管理者可删除任意问题。
- 被采纳的回答不能由其作者删除（防“骗采纳再删除”），课程管理者可删除；路人不能编辑或删除他人内容（403）。
- 不能给自己的内容投票（422 `QA-1015`）；评论回复只允许一层，且必须挂在同一目标上。

3.4.3 Reputation 声誉上下文

构造块	类型	要点	待实现方法
`ReputationAccount`	聚合根	reputation（= 流水之和）、availablePoints、frozenPoints、version	`open applyReputation freezePoints spendFrozen refundFrozen ensureFrozen requirePositive`

构造块	类型	要点	待实现方法
Bounty	聚合根（状态机）	OPEN → （采纳）AWARDED；OPEN → （到期/发起人自答被采纳）EXPIRED；终态不可变	open award close isDue ensureOpen
LedgerEntry	值对象（record）	只追加流水；幂等键 (eventId, userId, reason)	—
ReputationRulebook	领域策略	+15 / +10 / −2；差值计分；每日上限 200；开户积分	voteEffect voteDelta applyDailyCap initialPoints
BadgePolicy	领域策略（纯函数）	按采纳数、声誉阈值决定应得徽章	forAcceptedCount forReputation
AcademicTerm	领域服务（静态）	校园时区 Asia/Shanghai；学期/ISO 周/月 键	termKey weekKey monthKey

3.4.4 Discovery 内容发现上下文

构造块	类型	要点	待实现方法
SearchQuery	值对象	清洗全文检索运算符、≤50 字符、词项 ≥2 字符、≤5 个词项、默认排序推断	terms()、terms(String)
SearchDocument / SearchHit	值对象	搜索投影的写模型 / 读结果	—
RelatedQuestionPolicy	领域策略	J(A,B)=|A∩B|/|A∪B|；同分时得分高、已采纳优先	jaccard rank
Notification	聚合根	接收者、类型、标题、链接、来源事件、已读	create markRead
MentionParser	领域服务	正则提取 @昵称，去重保序，最多 5 个	extract
SearchPort / SearchIndex / NotificationRepository	端口 / 仓储接口	MySQL FULLTEXT 与 LIKE 两个适配器可配置切换	—

3.5 领域事件目录

事件	发布者（触发用例）	订阅者及处理
UserRegisteredEvent	Identity：注册 / 创建教职工	Reputation：开立声誉账户并按角色发放初始悬赏积分
QuestionPostedEvent	Q&A：发布问题	Discovery：插入搜索投影
QuestionEditedEvent	Q&A：编辑问题	Discovery：更新投影文本与标签

事件	发布者（触发用例）	订阅者及处理
`AnswerSubmittedEvent`	Q&A：提交回答	Reputation：授予“初来乍到”；Discovery：投影回答数 +1、通知提问者
`AnswerAcceptedEvent`	Q&A：采纳	Reputation：+15、徽章、悬赏结算；Discovery：标记已采纳、通知回答者
`AnswerEndorsedEvent`	Q&A：教师认证	Reputation：授予“名师认可”；Discovery：通知回答者
`VoteCastEvent`	Q&A：投票/改投/撤销	Reputation：按差值计分（受每日上限约束）；Discovery：问题得分同步
`CommentPostedEvent`	Q&A：评论	Discovery：解析 @提及并通知
`ContentDeletedEvent`	Q&A：软删除问题/回答	Discovery：标记投影删除 / 回答数 −1
`ReputationChangedEvent`	Reputation：记账	Reputation 排行榜：更新周/月/学期 ZSET（经 ProcessedEventStore 幂等）
`BadgeAwardedEvent`	Reputation：授予徽章	Discovery：通知获得者
`BountyOpenedEvent` / `BountyAwardedEvent` / `BountyClosedEvent`	Reputation：悬赏发起/结算/关闭	Discovery：维护“悬赏中”标记；`BountyAwarded` 通知获奖者

3.6 战术设计编码约定

以下约定均可从 Identity 参考实现中找到原型，所有 PR 的 Code Review 都按此检查：

约定	做法	反例
工厂方法创建聚合	`Question.post(...)`、`Answer.submit(...)`、`Bounty.open(...)`：在工厂中完成全部业务校验，`id` 为 null	在应用服务里 new 一个“半成品”再逐个 set
重建不校验	`reconstitute(...)` 只供仓储使用，原样还原状态	在 reconstitute 中重复业务校验导致历史数据无法加载
值对象自校验	`record` 的紧凑构造器内规范化 + 校验（参考 `Title`、`TagName`）	把长度校验散落在控制器和服务里
显式传入操作者	领域方法接收 `Actor` 与 `actorManagesCourse`，由应用服务查询 `IdentityApi` 后传入	领域层调用 `SecurityContextHolder` 或注入 `IdentityApi`
异常带错误码	`throw new ForbiddenException("QA-3001", "只有提问者或课程教师/助教可以采纳")`	抛 `IllegalArgumentException` 或返回 boolean

约定	做法	反例
计数器不回写	`score/answerCount/viewCount` 只读；变化由 `adjustXxx` 原子 SQL 完成（ADR-008）	`question.setAnswerCount(question.answerCount()+1)` 然后 save
时间可测试	应用服务通过注入的 `Clock` 获取 `clock.instant()`，传入领域方法 `now`	领域层直接 `Instant.now()`
应用服务编排模板	加载 → （查授权）→ 领域方法 → 保存 → 原子计数 → 发布事件 → 失效缓存 → 审计	在应用服务里写 if-else 业务规则（“贫血模型”）

代码清单 3-1 应用服务编排骨架示意（字段与参数以实际代码和测试为准）

```
@Transactional
public void accept(Actor actor, long questionId, long answerId) {
    Question question = loadQuestion(questionId);                // 1 加载聚合（不存在 → 404）
    Answer answer = loadAnswer(answerId);
    boolean manages = identity.canManageCourse(actor.userId(), question.courseId()); // 2 查授权
    question.accept(answer, actor, manages);                      // 3 领域规则在聚合内
    questions.save(question);                                     // 4 保存
    events.publish(new AnswerAcceptedEvent(EventIds.next(), clock.instant(), /* … */)); // 5 发件箱
    audit.record("ANSWER_ACCEPTED", actor.userId(), "ANSWER", String.valueOf(answerId), null); // 6 审计
    listCache.invalidateAll();                                    // 7 失效缓存
}
```

3.7 错误码约定

`docs/错误码.md` 已给定的错误码必须严格使用；未给定的场景，建议按下表补充，并在 Sprint 0 由全组确认后写回 `docs/错误码.md`（这本身就是统一语言的一部分）。

错误码	异常类型	场景	来源
QA-1001 / QA-1002	BusinessRule	标题长度 / 正文长度不合法	已给定
QA-1004 / QA-1005	BusinessRule	标签格式 / 标签数量 1–5	已给定（代码已实现）
QA-1015	BusinessRule	不能给自己的内容投票	已给定（集成测试断言）
QA-2002	Conflict	问题已采纳，不能重复采纳	已给定
QA-3001	Forbidden	只有提问者或课程教师/助教可以采纳	已给定
QA-1003	BusinessRule	评论长度不合法（建议 2–600 字符，与 `@Size(max=600)` 一致）	建议
QA-1010 / QA-1011	BusinessRule	答案不属于该问题 / 已删除的回答不能被采纳	建议

错误码	异常类型	场景	来源
QA-1012 / QA-1013	BusinessRule	被采纳的回答不能由作者删除 / 已有回答的问题提问者不能删除	建议
QA-1014	BusinessRule	投票值只能为 −1、0（撤销）、+1	建议
QA-1016 / QA-1017	BusinessRule	回复只允许一层 / 回复必须与父评论同一目标	建议
QA-1020	BusinessRule	内容已删除，不能回答/编辑	建议
QA-2003	Conflict	该回答已被教师认证	建议
QA-3002 / QA-3003 / QA-3004	Forbidden	无权编辑 / 无权删除 / 无权认证	建议
QA-4001 / QA-4002 / QA-4003	NotFound	问题 / 回答 / 评论不存在或已删除	建议
REP-1002	BusinessRule	可用悬赏积分不足	已给定
REP-1003 / REP-1004	BusinessRule	悬赏积分 10–500 / 期限 1–14 天超范围	已给定
REP-3002	Forbidden	只能查看自己的声誉流水	已给定
REP-1001 / REP-1005 / REP-1006	BusinessRule	问题已删除或已采纳不能悬赏 / 发起人不能赢得自己的悬赏 / 积分必须为正	建议
REP-2001 / REP-2002	Conflict	该问题已有进行中的悬赏 / 悬赏已结束	建议
REP-3001	Forbidden	仅课程管理者可以发起悬赏	建议
REP-4001 / REP-4002	NotFound	声誉账户 / 问题不存在	建议
DIS-1001 / DIS-1002	BusinessRule	词项至少 2 字符 / 关键词 ≤50 字符	已给定（代码已实现）
DIS-3001 / DIS-4001	Forbidden / NotFound	只能标记自己的通知 / 通知不存在	建议

第 4 章 团队组织与 Scrum 框架

4.1 Scrum 角色

角色	由谁担任	职责
产品负责人（PO）	团队的产品经理（建议成员 B）；任课教师作为主要干系人（客户）	维护 Product Backlog 排序、编写验收标准、在评审会上代表团队接受/拒绝 PBI；**在本地 IntelliJ 导入起始代码并创建团队仓库（PO 个人账号下）**，邀请成员为协作者，配置分支保护与看板。本项目不另设仓库管理员
Scrum Master（SM）	组内成员，**每个 Sprint 轮换**	主持各项 Scrum 事件、维护看板与燃尽图、清除障碍（环境、冲突、CI 红灯）、监督工作约定
开发者（Developers）	全体成员（含 PO 与 SM）	自组织完成 Sprint Backlog；每人每个开发 Sprint 至少交付 1 个领域层 PR 与 1 个应用层 PR

5 人组建议分工（“主责”指技术把关与 Code Review 第一责任人，并非独占编码；4 人组将 C、D 合并）：

成员	主责上下文 / 方向	横向职责	各 Sprint 的焦点
A	Q&A 问题与查询	Sprint 1 的 SM	S1 问题聚合与命令/查询；S2 账户事件处理；S3 搜索投影
B	Q&A 回答/采纳 → Reputation 计分	PO（产品经理）+ 仓库所有者	S0 建库与协作设置、Backlog 导入；S1 回答与采纳；S2 计分规则与账本；S3 通知投影
C	投票评论 → 悬赏 → 搜索推荐	Sprint 2 的 SM	S1 投票/评论；S2 悬赏；S3 搜索与相关问题
D	前端 + 排行榜 + 通知中心	Sprint 3 的 SM，前端负责人	每个 Sprint 的前端故事；S2 排行榜/学期；S3 通知服务
E	质量与 DevOps（CI、ArchUnit、测试替身、集成测试）	Sprint 0/4 的 SM，CI 流水线负责人	S0 CI 门禁与开发者须知；S1 行走骨架；S2/S3 应用层测试与端到端贯通

4.2 Scrum 事件与时间盒

事件	时间点	时间盒	输入 → 输出
Sprint 计划会	每个 Sprint 第 1 天（实验课）	2 周 Sprint ≤ 2 h；1 周 Sprint ≤ 1 h	排序后的 Product Backlog + 团队容量 → Sprint Goal + Sprint Backlog（任务拆到 ≤ 1 天）

事件	时间点	时间盒	输入 → 输出
每日站会	每周至少 3 次（线下 15 分钟，或在群里/Issue 按模板异步）	15 min	三问：昨天完成了什么？今天计划做什么？遇到什么阻碍？→ 更新看板、暴露阻碍
Backlog 精化	每个 Sprint 中段	≤ 1 h	下个 Sprint 候选 PBI → 拆分、补验收标准、Planning Poker 估算，达到 DoR
Sprint 评审会	Sprint 最后 1 天（实验课，教师参加）	≤ 1 h（每组 15–20 min 演示）	可运行增量 + GitHub 证据 → 教师反馈、PBI 接受/退回、Backlog 调整
Sprint 回顾会	评审会之后	≤ 45 min	本 Sprint 数据（燃尽、PR 周转、CI 红灯次数）→ 1 ~ 3 条可执行改进项（建 Issue 跟踪）

4.3 Scrum 工件及其在 GitHub 上的载体

工件	GitHub 载体	维护规则
Product Backlog	GitHub Projects（Board/Table 视图）中所有带 `type:story` 的 Issue；自定义字段 Priority、Story Points、Sprint	PO 负责排序；表格视图按 Rank 字段排序即为优先级顺序
Sprint Backlog	当前 Milestone（如 `Sprint 1`）中的 Story Issue + 拆出的 Task Issue（或 Task List 复选框）	只能由开发者增删任务；Sprint Goal 写在 Milestone 描述中
Increment（增量）	`main` 分支上的版本标签（v0.1.0 … v1.0.0）+ GitHub Release + CI 构建的镜像	必须满足 DoD；评审会现场从该标签启动演示
燃尽图	Projects 的 Insights（按 Story Points 统计）或团队自制表格	SM 每次站会后更新

4.4 就绪定义（DoR）与完成定义（DoD）

4.4.1 Definition of Ready（PBI 可以进入 Sprint 的条件）

- 以“作为…我希望…以便…”格式书写，价值明确；关联史诗（Epic）与限界上下文标签。
- 有可验证的验收标准（Given/When/Then），并已标出对应的已有测试类或需新增的测试。
- 已估算故事点（≤ 8 点；超过则拆分），依赖已识别（如需要上游事件字段）。

4.4.2 Definition of Done（PBI 完成的条件，全部满足才可在评审会上被接受）

1. 代码经 Pull Request 合入 `develop`，至少 1 名非作者成员 Approve，PR 关联并关闭对应 Issue。

2. 相关领域测试与应用层测试全部通过；新增业务规则有对应单元测试；**未修改任何已有断言。**
3. ArchUnit 架构测试通过（无框架侵入领域层、无跨上下文越界、无环）。
4. 所在领域模块行覆盖率 ≥ 70%（按 8.3 节的渐进门禁执行）。
5. CI 流水线在 PR 与合并后的 `develop` 上均为绿色。
6. 如涉及接口或错误码：Swagger UI 中可见且可调用，错误码符合 3.7 节约定；如涉及前端：`npm run typecheck && npm run build` 通过。
7. 在本地 `docker compose` 环境中按验收标准手工演示通过（附截图或 GIF 到 PR 描述）。

4.5 估算与容量

- 故事点采用斐波那契数列（1、2、3、5、8），以“PB-11 投票/改投/撤销 = 5 点”为基准故事进行相对估算；使用 Planning Poker（可用 IntelliJ 插件或在线工具）。
- 容量估算：每位同学每周课外投入约 8 ~ 10 小时；2 周 Sprint 的 5 人组容量约 80 ~ 100 理想小时，计划时只承诺 80%。
- 本指导书给出的故事点与任务工时是**参考值**，各组应在 Sprint 0 重新估算，并从 Sprint 2 起用上个 Sprint 的实际速度（Velocity）做计划。

第 5 章 Product Backlog

本章给出完整、已排序的 Backlog，各组在 Sprint 0 导入 GitHub Projects 后可按自身情况微调。

5.1 用户角色

角色	描述	关键诉求
访客	未登录用户	浏览问题、搜索、查看排行榜
学生（STUDENT）	自助注册，经管理员认证后显示认证徽章	提问、回答、投票、评论、获得声誉与徽章
助教（TA）/ 教师（TEACHER）	管理员创建；课程内为教师或助教时即“课程管理者”	代为采纳、教师认证、发起悬赏、删除不当内容
管理员（ADMIN）	平台运维	认证用户、创建教职工、调整角色、查看对账、禁用账号

5.2 史诗（Epics）

编号	史诗	范围
E0	协作与交付基础设施	仓库、分支保护、看板、CI 门禁、行走骨架
E1	身份与课程（Identity）	注册登录、课程、管理功能与扩展
E2	课程问答（Q&A）	问题、回答、采纳、投票、评论、认证、删除、查询
E3	声誉与徽章（Reputation）	账户、账本、计分规则、徽章、排行榜、对账
E4	悬赏（Reputation）	发起、冻结、结算、到期退款
E5	内容发现（Discovery）	搜索投影、全文检索、相关推荐、通知与提及
E6	质量与发布	测试、覆盖率、集成测试、可观测性、发布
E7	前端体验	各故事的界面补全与体验优化

5.3 排序原则

优先级采用 **MoSCoW + 依赖顺序 + 风险前置** 三条规则共同决定，表中的序号即最终排序：

1. **依赖优先**：事件链路决定了天然顺序——Q&A 是所有事件的源头，Reputation 的悬赏依赖 `QaQueryApi`，Discovery 同时订阅 Q&A 与 Reputation 的事件，因此 Q&A → Reputation → Discovery。

2. **价值优先**：“提问—回答—采纳”闭环是最小可用产品（MVP），其次是让社区活跃起来的激励体系，再次是提升效率的发现能力。
3. **风险前置**：协作基础设施、CI 门禁、发件箱阻塞问题（行走骨架）在最早的 Sprint 中处理；端到端集成测试在 Sprint 3 打通，为 Sprint 4 留出缓冲。
4. **同一 Sprint 内**：Must 先于 Should；领域层故事先于依赖它的前端故事；Could 项只在容量富余时拉入。

5.4 完整 Product Backlog（已排序）

#	ID	用户故事	史诗	优先级	SP	Sprint	实现落点
1	PB-01	作为团队，我们需要由 PO 创建的共享 GitHub 仓库、受保护分支、模板与看板，以便所有代码经评审后集成	E0	Must	3	S0	PO 个人仓库设置、`.github/`
2	PB-02	作为开发者，我需要一键启动 MySQL/Redis/后端/前端并跑通测试，以便在一致的环境开发	E0	Must	2	S0	`deploy/docker-compose.yml`
3	PB-03	作为团队，我们需要事件风暴产出的统一语言与上下文映射，以便沟通和编码一致	E0	Must	3	S0	`docs/ddd/`
4	PB-04	作为团队，我们需要渐进式 CI 门禁，以便每个 Sprint 结束时流水线都是绿的	E0	Must	2	S0	`.github/workflows/ci.yml`、ADR-009
5	PB-05	作为学生，我可以注册、登录并在课程页面浏览和加入课程，以便在课程问答区提问	E1	Must	3	S0	Identity（已实现）+ 前端课程页
6	PB-06	作为学生，我可以发布带标签、可关联课程的问题，以便向同学和老师求助	E2	Must	5	S1	`Question.post`、值对象、`QuestionCommandService.post`
7	PB-07	作为用户，我可以分页浏览（按课程筛选）问题列表，以便发现待解决的问题	E2	Must	3	S1	`QuestionQueryService.list`
8	PB-08	作为用户，我可以查看问题详情（渲染后的正文、回答、评论、我的投票、可用操作）	E2	Must	5	S1	`QuestionQueryService.detail`
9	PB-09	作为学生，我可以回答未删除的问题，以便帮助提问者	E2	Must	3	S1	`Answer.submit`、`AnswerCommandService.submit`
10	PB-10	作为提问者或课程管理者，我可以采纳一个最佳答案，以便标记问题已解决	E2	Must	5	S1	`Question.accept`、`AnswerCommandService.accept`
11	PB-11	作为用户，我可以对他人的问题/回答投票、改投或撤销，以便评价内容质量	E2	Must	5	S1	`Vote`、`VoteService`
12	PB-12	作为作者或课程管理者，我可以编辑问题和回答，以便修正内容	E2	Must	3	S1	`edit` 方法

#	ID	用户故事	史诗	优先级	SP	Sprint	实现落点
13	PB-13	作为作者或课程管理者，我可以软删除问题和回答，以便清理不当内容且可追责	E2	Must	3	S1	`softDelete`、`delete` 用例
14	PB-14	作为课程教师/助教，我可以为优质回答做“教师认证”，以便给学生权威指引	E2	Should	2	S1	`Answer.endorse`、`AnswerCommandService.endorse`
15	PB-15	作为用户，我可以评论问题或回答并进行一层回复，以便追问与澄清	E2	Must	3	S1	`Comment.post`、`CommentService`
16	PB-16	作为用户，我可以看到热门标签，以便按主题浏览	E2	Should	1	S1	`popularTags`、`Tags.names`
17	PB-17	作为其他上下文，我可以通过 QaQueryApi 查询问题引用，以便悬赏与通知使用	E2	Must	1	S1	`QaQueryApiImpl.findQuestion`
18	PB-18	作为团队，我们需要行走骨架（未实现监听器暂为空实现），以便发件箱不被阻塞	E0	Must	1	S1	Reputation/Discovery 的 `@EventListener`
19	PB-19	作为用户，我可以在前端编辑/删除内容、发表评论与回复，并看到友好的错误提示	E7	Should	3	S1	`frontend/src/pages/*`
20	PB-20	作为新用户，系统为我开立声誉账户（教师获得初始悬赏积分），以便参与激励体系	E3	Must	3	S2	`ReputationAccount.open`、`initialPoints`、`on(UserRegistered)`
21	PB-21	作为回答者，回答被采纳时我获得 +15 声誉（自问自答不加分）并记入账本	E3	Must	5	S2	`on(AnswerAccepted)`、`applyReputation`
22	PB-22	作为作者，被赞 +10、被踩 −2，改投/撤销按差值计算，且每日正向投票所得 ≤ 200	E3	Must	5	S2	`voteEffect/voteDelta/applyDailyCap`、`on(VoteCast)`
23	PB-23	作为用户，我在达成里程碑时自动获得徽章	E3	Should	3	S2	`BadgePolicy`、`BadgeService.grant`、其余事件处理
24	PB-24	作为课程管理者，我可以为问题发起悬赏（10–500 分，1–14 天）并冻结积分	E4	Must	5	S2	`Bounty.open`、`freezePoints`、`BountyService.open/latest`
25	PB-25	作为回答者，问题采纳时我自动赢得悬赏；作为发起人，到期未采纳时积分自动退回	E4	Must	5	S2	`award/close/isDue`、`settleOnAccept/expireDue`
26	PB-26	作为用户，我可以查看周/月/学期/总排行榜	E3	Should	5	S2	`AcademicTerm`、`LeaderboardService`
27	PB-27	作为用户，我可以查看他人声誉档案与徽章，并查看自己的积分与声誉流水	E3	Must	3	S2	`ReputationQueryService`、`ReputationApiImpl`
28	PB-28	作为管理员，我可以查看每日对账结果，以便发现声誉异常（FF-8）	E3	Should	2	S2	`ReconciliationService`
29	PB-29	作为教师，我可以在对话框中发起悬赏并看到倒计时；作为用户，我有个人声誉主页	E7	Should	3	S2	前端悬赏对话框、声誉页
30	PB-30	作为团队，我们需要 Reputation 应用层单元测试（手写 Fake），以便安全重构	E6	Must	3	S2	`co-reputation-application/src/test`

#	ID	用户故事	史诗	优先级	SP	Sprint	实现落点
31	PB-31	作为用户，我可以用中英文关键词搜索并按标签/未回答/悬赏筛选、按相关度/最新/活跃/热度排序	E5	Must	5	S3	`SearchQuery.terms`、`DiscoveryQueryService.search`
32	PB-32	作为系统，搜索投影随问答与悬赏事件幂等更新，以便搜索结果及时准确	E5	Must	5	S3	`SearchIndexProjector`
33	PB-33	作为读者，我在问题详情页看到相关问题推荐	E5	Should	3	S3	`RelatedQuestionPolicy`、`related`
34	PB-34	作为用户，我会收到新回答、被采纳、被认证、赢得悬赏、获得徽章等通知	E5	Must	5	S3	`Notification.create`、`NotificationProjector`
35	PB-35	作为用户，评论中被 @ 时我会收到提醒（每条最多 5 人）	E5	Should	3	S3	`MentionParser`、`on(CommentPosted)`
36	PB-36	作为用户，我可以查看通知列表、未读数，并标记单条或全部已读	E5	Must	3	S3	`NotificationService`、`markRead`
37	PB-37	作为团队，端到端集成测试全绿且领域覆盖率 ≥ 70%，以便发布候选版本	E6	Must	3	S3	`CampusOverflowFlowIT`、JaCoCo
38	PB-38	作为演示者，demo profile 能加载完整演示数据	E6	Should	1	S3	`DemoDataSeeder`
39	PB-39	作为团队，我们需要 Discovery 应用层单元测试（验证投影幂等）	E6	Must	3	S3	`co-discovery-application/src/test`
40	PB-40	作为管理员，我可以禁用违规账号并使其立即下线（ADR-003）	E1	Should	5	S4	`User.disable`（已有）+ 服务/接口/会话清理
41	PB-41	作为用户，我可以查看和修改个人资料（学院、班级）	E1	Could	3	S4	Identity 扩展
42	PB-42	作为用户，我获得更好的前端体验：分页/骨架屏/错误重试、Markdown 实时预览	E7	Should	3	S4	前端
43	PB-43	作为用户，通知红点实时刷新（SSE 替代 30 秒轮询）	E7	Could	5	S4	前端 + 新接口
44	PB-44	作为运维者，我可以在 Grafana 看到发件箱积压、死信、对账不一致等指标	E6	Should	2	S4	Prometheus/Grafana
45	PB-45	作为团队，我们对问题列表做压测并验证缓存命中	E6	Could	3	S4	压测脚本
46	PB-46	作为干系人，我可以用 docker compose 一键部署 v1.0 并阅读 Release Notes	E6	Must	2	S4	GitHub Release、镜像
47	PB-47	作为管理员，我有管理页面：认证学生、创建教职工账号	E1	Could	3	S4	前端 + 已有接口

#	ID	用户故事	史诗	优先级	SP	Sprint	实现落点
48	PB-48	作为教师，我可以审批学生的选课申请	E1	Won't（本期）	8	—	Identity 扩展（留作后续）

各 Sprint 计划故事点：S0 = 13，S1 = 43，S2 = 42，S3 = 31，S4 = 26（其中 Must/Should = 12，其余 Could 项按剩余容量选做）。

> **要点｜把 Backlog 导入 GitHub**
>
> 每个 PBI 建一个 Issue（使用附录 B 的 User Story 模板），标题形如 `[PB-10] 采纳最佳答案`，打上 `type:story`、`ctx:qa`、`priority:must` 标签，设置 Story Points 与 Sprint 字段，并加入 Project。
>
> 可在 Sprint 0 由 PO 用 `gh issue create` 脚本批量导入（示例见 7.4 节），再在表格视图中拖拽排序。

5.5 依赖关系

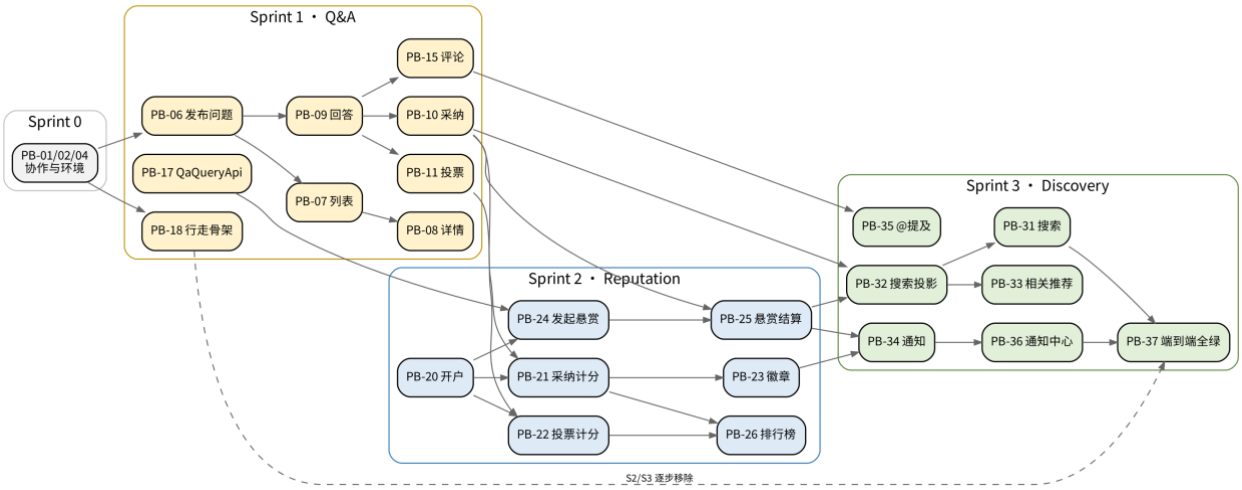


图 5-1 PBI 依赖图（虚线：行走骨架在 S2/S3 逐步移除）

第 6 章 Sprint 规划与执行指南

6.1 Sprint 总览

Sprint	时间	Sprint Goal	纳入 PBI	验收的可运行软件
S0	第 1 周	团队、仓库、环境、CI 与 Backlog 全部就绪，能注册/登录并加入课程	PB-01 ~ 05	注册→登录→课程列表→加入课程；CI 绿；看板就绪
S1	第 2–3 周	学生可以完成"提问→回答→投票→评论→采纳"的完整问答闭环	PB-06 ~ 19	前端完成问答闭环；Q&A 领域+应用测试全绿
S2	第 4–5 周	问答行为能异步、幂等地转化为声誉、徽章、悬赏与排行榜	PB-20 ~ 30	采纳后声誉 +15、徽章、悬赏结算/退款、排行榜
S3	第 6–7 周	用户能搜到问题、看到相关推荐并收到通知，端到端链路全绿	PB-31 ~ 39	`mvn verify` 全绿；搜索、推荐、通知、@提及；demo 数据
S4	第 8 周	系统稳定化、体验打磨、可观测，完成 v1.0 发布与最终答辩	PB-40 ~ 47（按容量）	docker compose 一键部署 v1.0；Grafana；最终演示

两周 Sprint 的标准节奏（以每周一次实验课为例）：

时间	活动
第 1 周实验课	Sprint 评审（上一 Sprint）→ 回顾 → Sprint 计划会（本 Sprint），在 Milestone 写下 Sprint Goal
第 1 周课外	按任务开 feature 分支，测试驱动实现；每天至少一次推送；站会（周一/三/五）
第 2 周中段	Backlog 精化（下一 Sprint）；功能冻结前完成主要 PR 合并
第 2 周后半	集成联调、修复缺陷、补测试、准备演示脚本；Sprint 最后一天前 `develop → main` 发布 PR 并打标签

6.2 Sprint 0（第 1 周）：启动与基础设施

> **要点 | Sprint Goal**
>
> 团队、仓库、环境、CI 与 Product Backlog 全部就绪；每位成员都能在本地启动系统、完成注册/登录/加入课程，并完成一次完整的"分支→PR→评审→合并"。

6.2.1 Sprint Backlog

任务	内容	PBI	模块 / 层	负责	工时
T0-01	PO 在本地 IntelliJ 导入起始代码并推送到 PO 个人 GitHub 仓库，创建 `develop` 分支，邀请成员与教师为协作者，按 7.1.3 节配置分支保护（含“不允许绕过”）与 Projects 看板（步骤见 7.1.1）	PB-01	PO 仓库	B	3
T0-02	经 PR 提交 Issue 模板（Story/Task/Bug）、PR 模板、CODEOWNERS；创建标签集、Milestones（Sprint 0 ~ 4）与看板自定义字段	PB-01	`.github/`	A	3
T0-03	每人接受协作邀请，在 IntelliJ 中从 PO 仓库克隆项目（7.1.2），配置与 GitHub 一致的提交邮箱；安装 JDK 21、Maven、Docker、Node 20；启动中间件；运行 `mvn clean test`（预期红）与前端；提交环境截图	PB-02	全体	全员	2×5
T0-04	改造 CI 为“渐进式质量门禁”（按 Sprint 范围运行单元测试/覆盖率，集成测试暂允许失败），撰写 `docs/adr/009-渐进式质量门禁.md`	PB-04	CI	E	3
T0-05	精读 8 条 ADR 与 Identity 参考实现，每人负责讲解 1 ~ 2 条 ADR + 一层代码，合成《架构理解笔记》`docs/notes/architecture.md`	PB-03	文档	全员	2×5
T0-06	事件风暴工作坊（90 分钟）：领域事件→命令→聚合→策略→上下文边界，产出上下文映射图与统一语言词汇表 `docs/ddd/`	PB-03	文档	全员	2×5
T0-07	Product Backlog 导入 GitHub（48 个 Issue），Planning Poker 重新估算，确认 DoR/DoD 与团队工作约定 `docs/team-agreement.md`	PB-01	Projects	B	3
T0-08	用 IntelliJ HTTP Client 编写 `http/auth.http`：csrf→register→login→me→courses→join，作为后续接口冒烟脚本	PB-05	`http/`	C	2
T0-09	前端新增“课程”页：课程列表、我的课程、加入课程；提问页可选择课程	PB-05	frontend	D	5
T0-10	破冰 PR：每人在 `docs/team.md` 写入个人信息与职责，走完整 PR 流程（含至少 1 条有效评审意见）	PB-01	文档	全员	0.5×5
T0-11	准备教师/管理员测试账号的获取方式（见 6.2.3），写入 README“开发者须知”	PB-02	文档	E	1

6.2.2 关键操作步骤

代码清单 6-1 Sprint 0 环境验证

```
# 1) 克隆并启动中间件
git clone git@github.com:<po-username>/campus-overflow.git && cd campus-overflow   # 或 IntelliJ: Get from VCS
docker compose -f deploy/docker-compose.yml up -d mysql redis

# 2) 后端：编译 + 测试（第一次会有约 38 个失败，属于预期）
cd backend && mvn clean test
mvn spring-boot:run -pl co-bootstrap -Dspring-boot.run.profiles=dev   # 不要加 demo（依赖未实现的业务）

# 3) 前端
cd ../frontend && npm install && npm run dev         # http://localhost:5173
# 4) 接口文档：http://localhost:8080/swagger-ui.html
```

6.2.3 获取教师/管理员账号（Sprint 1 ~ 2 开发期使用）

`DemoDataSeeder` 依赖悬赏等尚未实现的业务，Sprint 2 结束前无法使用 demo profile。开发期可先正常注册账号，再用 SQL 提升角色（**修改后需重新登录**，因为会话中缓存了 Actor）：

```
-- 进入容器：docker exec -it co-mysql mysql -ucampus -pcampus_dev_pwd campus_overflow
UPDATE id_user SET role = 'TEACHER', verified = TRUE WHERE username = 'T2001';
UPDATE id_user SET role = 'ADMIN',   verified = TRUE WHERE username = 'admin01';
-- 教师建课后，学生调用 POST /api/v1/courses/{id}/join 加入；
-- 教师可通过 PUT /api/v1/courses/{id}/members/{userId} 任命助教（role: TA）
```

6.2.4 验收标准

- 现场：任一成员的电脑上启动系统，完成注册→登录→加入课程；Swagger UI 可访问。
- GitHub：全体成员已作为协作者加入 PO 仓库并从中克隆；`main`/`develop` 已受保护且所有者也不能绕过；看板中 48 个 PBI 已排序且 Sprint 1 的 PBI 满足 DoR；每位成员至少 1 个已合并 PR 和 1 次评审。
- CI：`develop` 最新提交的 CI 为绿色（渐进门禁下运行共享内核与 Identity 测试）。
- 文档：`docs/ddd/`（上下文映射、统一语言、事件清单）、`docs/notes/architecture.md`、ADR-009 已合入。
- 在 `main` 打标签 `v0.1.0`。

6.3 Sprint 1（第 2–3 周）：问答核心

> **要点 | Sprint Goal**
>
> 学生可以在前端完成“提问 → 回答 → 投票 → 评论 → 采纳”的完整问答闭环，教师可以认证与删除内容；Q&A 领域测试与应用层测试全部通过。

6.3.1 Sprint Backlog

任务	内容	PBI	模块 / 层	负责	工时
T1-01	`MarkdownBody.ofQuestion/ofAnswer`、`Tags.names`（让 `QuestionTest.Posting` 全绿）	PB-06	qa-domain	A	2
T1-02	`Question.post/edit/ensureNotDeleted/ensureOpenForAnswers/softDelete`	PB-06/12/13	qa-domain	A	4
T1-03	`Question.accept`（`QuestionTest.Accepting` 全绿）	PB-10	qa-domain	B	3
T1-04	`Answer.submit/edit/endorse/softDelete/ensureNotDeleted`（`AnswerTest` 全绿）	PB-09/12/13/14	qa-domain	B	4
T1-05	`Vote.cast/change/checkValue`、`Comment.post`（`VoteAndCommentTest` 全绿）	PB-11/15	qa-domain	C	4
T1-06	补充领域边界测试：标题 9/10/150/151 字符、6 个标签、重复采纳、已删除问题编辑、二次认证等	PB-06 ~ 15	qa-domain test	全员	4
T1-07	`QuestionCommandService.post/edit/delete/load` + 新增 `QuestionCommandServiceTest`（复用 `QaTestFixtures`）	PB-06/12/13	qa-application	A	8
T1-08	`AnswerCommandService` 7 个方法（`AnswerAndVoteServiceTest` 全绿）	PB-09/10/13/14	qa-application	B	7
T1-09	`VoteService.cast/resolve`、`CommentService.post` + `CommentServiceTest`	PB-11/15	qa-application	C	7
T1-10	`QuestionQueryService.list/detail/popularTags` + 基于假读模型的测试	PB-07/08/16	qa-application	D	10
T1-11	`QaQueryApiImpl.findQuestion`	PB-17	qa-application	E	1
T1-12	行走骨架：Reputation/Discovery 全部 `@EventListener` 方法暂为空实现 + `tech-debt` Issue	PB-18	rep/dis-application	E	1
T1-13	前端：编辑/删除问题与回答、评论与回复、根据 `permissions` 显示按钮、按错误码提示	PB-19	frontend	D	6
T1-14	CI 门禁范围加入 Q&A 模块；`co-qa-domain` 覆盖率门槛设为 0.70	PB-04	CI / pom	E	2
T1-15	`http/qa.http` 冒烟脚本 + Sprint 评审演示脚本	—	`http/`	E	2

6.3.2 实现要点：领域层

方法	业务规则 / 实现要点	异常与错误码
`MarkdownBody.ofQuestion / ofAnswer`	复用 `Guard.requireText` 做非空与长度（按码点）校验；问题正文建议 20–30000、回答 10–30000（上限与控制器 `@Size(max=30000)` 一致，下限需兼容测试样例文本长度，由团队统一后写入错误码文档）	QA-1002
`Tags.names`	返回规范化后的标签字符串列表，保持去重后的插入顺序：`#Java, java, Spring-Boot` → `[java, spring-boot]`	—
`Question.post`	工厂方法：id 为 null、`createdAt = updatedAt = now`、计数器为 0、未采纳、未删除、version 0；值对象已自校验	—

方法	业务规则 / 实现要点	异常与错误码
`Question.edit`	先 `ensureNotDeleted`；作者本人或 `actorManagesCourse` 才能编辑，否则 403；替换 title/body/tags，`updatedAt = now`	QA-1020 / QA-3002
`Question.accept`	顺序：未删除 → 有权限（作者或管理者，否则 403）→ 未采纳过（否则 409）→ 答案属于本题（`answer.questionId() == id`）→ 答案未删除；最后 `acceptedAnswerId = answer.id()`	QA-3001 / QA-2002 / QA-1010 / QA-1011
`Question.softDelete`	管理者可删任意问题；作者仅当 `answerCount == 0`（否则 422）；其他人 403；设置 `deletedAt = now`	QA-1013 / QA-3003
`ensureOpenForAnswers / ensureNotDeleted`	已删除则抛业务异常（已采纳的问题仍可继续回答）	QA-1020
`Answer.submit`	先调用 `question.ensureOpenForAnswers()`；`questionId = question.id()`；score 0、未认证	QA-1020
`Answer.edit / endorse`	edit：未删除 + 作者或管理者；endorse：未删除 + 管理者（否则 403）+ 未认证过（否则 409），`endorsedBy = actor.userId()`	QA-3002 / QA-3004 / QA-2003
`Answer.softDelete`	作者：若 `accepted` 为 true 则 422，否则可删；管理者：可删；其他人：403	QA-1012 / QA-3003
`Vote.cast / change / checkValue`	`voterId == targetAuthorId` → 422，消息必须包含"自己"；值只能为 ±1；`change` 返回旧值并更新 `updatedAt`	QA-1015 / QA-1014
`Comment.post`	正文 2–600 字符；若有 parent：parent 必须是顶层评论（`parentId == null`），且 targetType、targetId 与本评论相同	QA-1003 / QA-1016 / QA-1017

6.3.3 实现要点：应用层

方法	业务规则 / 实现要点	异常与错误码
`QuestionCommandService.post`	若指定 courseId，用 `identity.findCourse` 校验存在；构造值对象 → `Question.post` → save → `tags.adjustUsage(names, +1)` → 发布 `QuestionPostedEvent`（plainText 用 `renderer.toPlainText(body, PLAIN_TEXT_FOR_INDEX)`）→ `listCache.invalidateAll()`	ID-4002 / QA-4001
`QuestionCommandService.edit`	计算新旧标签差集（类中已 import `HashSet/Set`）：移除的 −1、新增的 +1；发布 `QuestionEditedEvent`；失效缓存	—
`QuestionCommandService.delete`	softDelete → save → 标签计数 −1 → `ContentDeletedEvent("QUESTION", id, id)` → 审计 `QUESTION_DELETED` → 失效缓存	—
`load / loadQuestion / loadAnswer`	不存在或已软删除统一抛 404（ADR-006 与错误码文档一致）	QA-4001 / QA-4002
`AnswerCommandService.submit`	Answer.submit → save → `questions.adjustAnswerCount(qid, +1)`（**测试断言原子增量**）→ 发布 `AnswerSubmittedEvent`（含 `questionAuthorId`、标题）→ 失效缓存**恰好一次**	—
`AnswerCommandService.accept / endorse`	见代码清单 3-1；采纳事件含 `answerAuthorId`、`questionAuthorId`、`acceptedById`；审计动作名必须为 `ANSWER_ACCEPTED`	—

方法	业务规则 / 实现要点	异常与错误码
`AnswerCommandService.delete`	`answer.softDelete(actor, manages, question.isAccepted(answerId), now)` → 回答数 -1 → `ContentDeletedEvent("ANSWER", aid, qid)` → 审计	—
`VoteService.cast`	value ∉ {-1,0,1} → 422；`resolve` 得到目标作者与 questionId；旧值 old（无则 0）；**old == value 时直接返回 (value, 0) 且不发事件**（幂等）；value 为 0 删除投票，否则改投或 `Vote.cast` 新建；delta = value − old，调用对应仓储 `adjustScore`；发布 `VoteCastEvent(old, value)`	QA-1014 / QA-1015
`VoteService.resolve`	QUESTION → 问题作者、自身 id；ANSWER → 回答作者、其 questionId；目标不存在或已删除 → 404	QA-4001 / QA-4002
`CommentService.post`	解析目标得到 questionId；parentId 非空则加载父评论（不存在 404）；`Comment.post` → save → 发布 `CommentPostedEvent`（含正文，供 @提及解析）	QA-4003
`QuestionQueryService.list`	先查缓存；未命中则读模型分页 → 批量 `identity.findUsers` → 摘要 `toPlainText(body, 160)`、`accepted = acceptedAnswerId != null` → 回填缓存	—
`QuestionQueryService.detail`	读模型取问题（无则 404）→ `viewCounter.increment` → 回答与评论 → 一次性收集所有用户 id 批量查询 → 登录时查 `findMyVotes`（key 形如 `ANSWER:50`）→ 计算 `PermissionsView` → 回答排序：已采纳优先、得分降序、时间升序 → 评论按目标分组	QA-4001
`popularTags` / `QaQueryApiImpl`	limit 限制在 1–50；`findQuestion` 映射为 `QuestionRef(id, authorId, courseId, title, acceptedAnswerId, deleted)`	—

6.3.4 行走骨架（Walking Skeleton）做法

代码清单 6-2 监听器临时空实现示例

```
// co-discovery-application/.../NotificationProjector.java (Sprint 1 临时实现，S3 替换)
@EventListener
@Transactional
public void on(AnswerSubmittedEvent e) {
    log.debug("[walking-skeleton] Discovery 尚未实现，忽略事件 {}", e.eventId()); // TODO(S3) 见 Issue
#42
}
```

- 只改 `@EventListener` 方法体，不改签名；每个文件顶部注释链接到 `tech-debt` Issue。Sprint 2 移除 Reputation 部分，Sprint 3 全部移除。
- 空实现期间事件会被标记为“已投递”。进入实现 Sprint 后若需重放：`docker compose down -v` 重建数据库，或执行 `UPDATE outbox_event SET published_at = NULL, attempts = 0;`（消费端幂等保证安全）。

6.3.5 验收标准与演示脚本

#	Given	When	Then
1	小明已登录	发布标题 ≥10 字、2 个标签的问题	201；首页列表第一条出现该问题，标签已小写去重
2	问题已存在	小红提交回答	201；问题回答数 +1；详情页显示回答（Markdown 已渲染且净化）
3	小红是普通学生	小红尝试采纳	403 `QA-3001`
4	小明是提问者	小明采纳小红的回答；再次采纳另一回答	第一次 204 并显示“已采纳”；第二次 409 `QA-2002`
5	小刚已登录	对回答连续点赞两次，再改为踩	得分依次为 1、1、−1；第二次点赞无变化
6	小刚是问题作者	给自己的问题点赞	422 `QA-1015`
7	回答下已有评论	回复该评论；再回复这条回复	第一次成功；第二次 422
8	王老师是课程教师	认证小红的回答；删除一个已有回答的问题	认证标记显示；删除成功，列表与详情不再出现

代码清单 6-3 Sprint 1 自检命令

```
mvn -f backend/pom.xml test -pl co-qa/co-qa-domain,co-qa/co-qa-application -am   # 全绿
mvn -f backend/pom.xml test -pl co-bootstrap -Dtest=ArchitectureTest              # 全绿
cd frontend && npm run typecheck && npm run build
```

6.4 Sprint 2（第 4–5 周）：声誉与激励

> **要点 | Sprint Goal**
>
> 问答行为能够异步、幂等地转化为声誉、徽章、悬赏与排行榜：采纳后回答者声誉 +15，悬赏可以发起、结算与到期退款，排行榜按周/月/学期展示，对账结果一致。

6.4.1 Sprint Backlog

任务	内容	PBI	模块 / 层	负责	工时
T2-01	`ReputationRulebook.voteEffect/voteDelta/applyDailyCap/initialPoints`（`ReputationRulebookTest` 全绿）	PB-20/22	rep-domain	B	2
T2-02	`ReputationAccount` 7 个方法	PB-20/24/25	rep-domain	B	3
T2-03	`Bounty.open/award/close/isDue/ensureOpen`	PB-24/25	rep-domain	C	3
T2-04	`BadgePolicy` 2 个方法 + `AcademicTerm` 3 个方法（`BountyAndAccountTest` 全绿）	PB-23/26	rep-domain	D	3

任务	内容	PBI	模块 / 层	负责	工时
T2-05	`BadgeService.grant`；`ReputationEventHandlers.on(UserRegistered / AnswerSubmitted / AnswerEndorsed)`	PB-20/23	rep-application	A	4
T2-06	`ReputationEventHandlers.on(AnswerAccepted / VoteCast)`（含每日上限与采纳数徽章）	PB-21/22	rep-application	B	5
T2-07	`BountyService.open/latest/settleOnAccept/expireDue/closeAndRefund`	PB-24/25	rep-application	C	9
T2-08	`LeaderboardService.on/top`	PB-26	rep-application	D	4
T2-09	`ReputationQueryService.profile/ledger`、`ReputationApiImpl.summary/reputations`	PB-27	rep-application	A	3
T2-10	`ReconciliationService.reconcile`	PB-28	rep-application	E	2
T2-11	新建 `ReputationTestFixtures`（内存仓储、RecordingEvents、StubIdentity、StubQa、固定 Clock）并编写应用层测试：重复事件不重复记账、每日上限、悬赏结算/自答退款/到期退款、对账	PB-30	rep-application test	E+ 各主责	10
T2-12	移除 Reputation 侧行走骨架；CI 范围加入 Reputation 模块；`co-reputation-domain` 覆盖率 ≥ 0.70	PB-18/04	CI	E	1
T2-13	前端：悬赏对话框（替换 `prompt()`）与倒计时；个人声誉主页（徽章墙、流水分页）；排行榜周期切换	PB-29	frontend	D	8
T2-14	验证 demo profile 可启动（Discovery 监听器仍为空实现）	PB-38	bootstrap	E	1

6.4.2 实现要点：领域层

方法	业务规则 / 实现要点	异常与错误码
`voteEffect / voteDelta`	effect：+1→+10，−1→−2，0→0；delta = effect(new) − effect(old)。例：赞改踩 = −2 − 10 = −12	—
`applyDailyCap(delta, gainedToday)`	delta ≤ 0 原样返回；否则返回 `max(0, min(delta, 200 − gainedToday))`：(10,195)→5，(10,200)→0，(−2,500)→−2	—
`initialPoints(role)`	TEACHER → 1000、STUDENT → 0（测试断言）；TA、ADMIN 也可能发起悬赏，建议 PO 决定（如 TA 500、ADMIN 1000）并记录到 ADR	—
`ReputationAccount.open`	reputation 0，available = initialPoints（允许 0，不能为负），frozen 0，version 0	REP-1006
`applyReputation(delta)`	reputation += delta；**不要设下限截断**，否则破坏“余额 = 流水之和”不变量（FF-8）	—
`freezePoints / spendFrozen / refundFrozen`	均先 `requirePositive`；freeze：points ≤ available 否则 422，available −= p，frozen += p；spend：`ensureFrozen` 后 frozen −= p；refund：`ensureFrozen` 后 frozen −= p、available += p	REP-1002 / REP-1006
`Bounty.open`	非管理者 → 403；points ∉ [10,500] → 422；days ∉ [1,14] → 422；`questionOpen == false` → 422；`expiresAt = now + days`，状态 OPEN	REP-3001 / REP-1003 / REP-1004 / REP-1001

方法	业务规则 / 实现要点	异常与错误码
award / close / isDue	award、close 先 ensureOpen（非 OPEN → 409）；award 时 winnerId == sponsorId → 422；isDue：OPEN 且 !now.isBefore(expiresAt)（到期时刻即视为到期）	REP-2002 / REP-1005
BadgePolicy	采纳数 ≥1 → FIRST_ACCEPTED，≥10 → HELPER_10；声誉 ≥100 → REPUTATION_100，≥1000 → REPUTATION_1000	—
AcademicTerm	先转换到 CAMPUS_ZONE；2–7 月 YYYY-SPRING，8–12 月 YYYY-FALL，1 月属上一年 (YYYY-1)-FALL；月键 2026-10；周键用 ISO 周年 + 周序号两位补零 2026-W41	—

6.4.3 实现要点：应用层（事件处理器运行在各自事务中）

方法	业务规则 / 实现要点	异常与错误码
on(UserRegisteredEvent)	账户已存在则忽略（幂等）；否则 ReputationAccount.open(userId, initialPoints(role)) 并保存	—
on(AnswerAcceptedEvent)	ledgerService.record(回答者, eventId, ANSWER_ACCEPTED, acceptedDelta(...), "ANSWER", answerId)；再用 ledger.countByUserAndReason 计算采纳数并 BadgePolicy.forAcceptedCount 授徽章；最后 bounties.settleOnAccept(...)	—
on(VoteCastEvent)	delta = voteDelta(old,new)；reason 按 targetType 取 QUESTION_VOTE / ANSWER_VOTE；delta > 0 时用 sumPositiveVoteDeltaSince(作者，当日 UTC 零点) 套用每日上限；投票人与作者相同不会出现（Q&A 已拒绝）	—
on(AnswerSubmitted / AnswerEndorsed)	分别授予 FIRST_ANSWER / TEACHER_ENDORSED（BadgeService.grant 本身幂等）	—
BadgeService.grant	badges.has 为真则返回；否则 award 并发布 BadgeAwardedEvent(code, displayName)	—
BountyService.open	qa.findQuestion（无 → 404）；questionOpen = 未删除且未采纳；manages = identity.canManageCourse；已有 OPEN 悬赏 → 409；Bounty.open → 发起人账户 freezePoints → 保存两者 → 发布 BountyOpenedEvent → 审计	REP-4002 / REP-2001
settleOnAccept	无 OPEN 悬赏直接返回（幂等）；获奖者 == 发起人 → closeAndRefund；否则 award → 发起人 spendFrozen → ledgerService.record(winner, sourceEventId, BOUNTY_AWARDED, points, "BOUNTY", bountyId) → 发布 BountyAwardedEvent → 审计	—
expireDue / closeAndRefund	findDue(now, 100) 逐个关闭；close → refundFrozen → 保存 → 发布 BountyClosedEvent(refundedPoints)；返回处理数量	—
LeaderboardService.on / top	on：processed.markProcessed(eventId, "reputation.leaderboard") 为真才 store.increment(userId, delta, occurredAt)；top：ALL 读 accounts.topByReputation，其余读 store.top(period, clock.instant(), limit)，批量取昵称并生成名次	—

方法	业务规则 / 实现要点	异常与错误码
`ReputationQueryService`	profile：他人查看时 availablePoints/frozenPoints 为 null；ledger：仅本人或管理员，否则 403	REP-3002
`ReconciliationService`	对比 `accounts.findAll()` 与 `ledger.sumByUser()`（无流水视为 0），返回不一致列表	—

> **注意 | 幂等是本 Sprint 的验收重点**
>
> 评审时教师可能会把某条 `outbox_event` 的 `published_at` 置空让其重放：声誉、徽章、悬赏、排行榜都不得重复变化。请在应用层测试中用“同一事件调用两次处理器”覆盖每一种事件。

6.4.4 验收标准

- 注册教师账号后其悬赏积分为 1000；学生为 0。
- 采纳后数秒内回答者声誉 = 15，流水中出现“回答被采纳 +15”，徽章“初露锋芒”；自问自答被采纳声誉不变。
- 被赞 +10、被踩 −2、赞改踩 −12；同日正向投票所得超过 200 的部分不计。
- 教师发起 50 分 / 7 天悬赏：可用 950、冻结 50；采纳他人回答后获奖者声誉 +50（BOUNTY_AWARDED）、发起人冻结归零；把 `expires_at` 改为过去并缩短 `co.reputation.bounty-expiry-interval-ms` 可演示到期退款。
- 排行榜周/月/学期/总榜可切换；`GET /api/v1/admin/reputation/reconciliation` 返回空列表；他人无法查看我的流水（403 `REP-3002`）。
- `mvn test` 覆盖 Q&A + Reputation 全部模块全绿；`main` 打标签 `v0.3.0`。

6.5 Sprint 3（第 6–7 周）：内容发现与端到端贯通

> **要点 | Sprint Goal**
>
> 用户可以用中文关键词搜到问题、看到相关问题推荐、收到各类通知与 @提醒；移除全部行走骨架，`mvn verify`（含 Testcontainers 端到端测试）在 CI 上全绿，发布候选版本 v0.9.0。

6.5.1 Sprint Backlog

任务	内容	PBI	模块 / 层	负责	工时
T3-01	`SearchQuery.terms()` 与 `terms(String)`	PB-31	dis-domain	C	1
T3-02	`RelatedQuestionPolicy.jaccard/rank`	PB-33	dis-domain	C	2
T3-03	`MentionParser.extract`、`Notification.create/markRead`（`DiscoveryDomainTest` 全绿）	PB-34/35/36	dis-domain	D	2

任务	内容	PBI	模块 / 层	负责	工时
T3-04	`SearchIndexProjector` 10 个方法	PB-32	dis-application	A	6
T3-05	`DiscoveryQueryService.search/related`	PB-31/33	dis-application	C	4
T3-06	`NotificationProjector` 8 个方法	PB-34/35	dis-application	B	6
T3-07	`NotificationService` 5 个方法	PB-36	dis-application	D	3
T3-08	Discovery 应用层测试：同一事件投递两次只生效一次、自己回答自己不通知、@ 不存在的昵称被忽略、他人不能标记已读	PB-39	dis-application test	E	6
T3-09	移除全部行走骨架；本地与 CI 运行 `mvn verify` 使 `CampusOverflowFlowIT` 全绿；CI 恢复完整门禁（ADR-009 状态改为“已完成”）	PB-37	bootstrap / CI	E	5
T3-10	根 POM `domain.coverage.minimum` 改为 0.70 并保证通过	PB-37	pom	E	2
T3-11	前端：搜索页筛选（标签/未回答/悬赏）与排序、相关问题侧栏、通知单条已读、评论中 @昵称高亮	PB-31/33/36	frontend	D	8
T3-12	用 demo profile 完整走查演示数据；全员回归测试与缺陷修复	PB-38	全体	全员	2×5

6.5.2 实现要点

方法	业务规则 / 实现要点	异常与错误码
`SearchQuery.terms`	实例方法：keyword 为 null 返回空列表；静态方法：按 `\s+` 切分、去空串、去重保序、最多 5 个。`"  +红黑树 -旋转*  "` → `[红黑树, 旋转]`	（构造器已校验 DIS-1001/1002）
`jaccard(shared, a, b)`	`shared / (a + b − shared)`，分母为 0 返回 0。`jaccard(2,3,3) = 0.5`	—
`rank(candidates, ownTagCount, limit)`	按 J 降序，再按 score 降序，再已采纳优先，截取 limit。测试样例结果为 `[2, 3]`	—
`MentionParser.extract`	用已给出的 `MENTION` 正则 `find()`，`LinkedHashSet` 去重保序，满 5 个停止；`a@b` 因昵称不足 2 字符不匹配	—
`Notification.create / markRead`	create：标题非空且截断到 200 字符、未读；markRead：`actorId != recipientId` → 403，重复标记幂等	DIS-3001
`SearchIndexProjector`	每个 on 先 `firstTime(e)`（`processed.markProcessed(eventId, CONSUMER)`）为假则返回。Posted → insert；Edited → updateText；AnswerSubmitted → 回答数 +1；ContentDeleted：QUESTION → markDeleted，ANSWER → 回答数 −1；Accepted → markAccepted；VoteCast 仅 QUESTION 目标 → adjustScore(new − old)；BountyOpened → bountyOpen=true；Awarded/Closed → false	—
`DiscoveryQueryService`	search：`SearchPort.search` → 批量取作者 → `SearchResultView`；related：`findTags` 为空返回空；`findCandidatesSharingTags(qid, tags, 50)` → `rank(..., tags.size(), limit)` → 带相似度的视图	—

方法	业务规则 / 实现要点	异常与错误码
`NotificationProjector`	AnswerSubmitted → 提问者（自答不通知）；AnswerAccepted/Endorsed → 回答者（自己操作自己不通知）；CommentPosted → 解析提及 → `findByDisplayName` → 跳过作者本人；BountyAwarded → 获奖者（标题经 `qa.findQuestion` 取得）；BadgeAwarded → 获得者。`notify` 先查 `exists(eventId, recipient)` 再保存；`name()` 查不到时返回“某同学”	—
`NotificationService`	list/unreadCount 按 `actor.userId()`；markRead：找不到 404，交给聚合校验接收者后保存；markAllRead 返回更新条数；`toView` 映射类型名	DIS-4001

提示 | 端到端测试排错清单

需要本机 Docker 可用（Testcontainers 拉取 `mysql:8.0.39` 与 `redis:7-alpine`）；IntelliJ 中可直接运行 `CampusOverflowFlowIT`。

声誉断言超时：查看 `outbox_event.last_error`，通常是某个监听器仍在抛异常或违反唯一约束。

搜索断言失败：确认 `QuestionPostedEvent` 的 plainText、ngram 参数和停用词设置；单字关键词会被 DIS-1001 拒绝。

6.5.3 验收标准

- 搜索“跳表”“红黑树”等中文词与英文词均可命中；按标签、未回答、悬赏筛选与四种排序生效；单字关键词返回 422 `DIS-1001`。
- 问题详情页显示相关问题及相似度；删除的问题不再出现在搜索与推荐中。
- 回答、采纳、认证、悬赏获奖、获得徽章、被 @ 都产生通知；未读数正确；只能标记自己的通知。
- CI 全部 5 个后端阶段 + 前端阶段为绿色，镜像构建成功；领域覆盖率 ≥ 70%；`main` 打标签 `v0.9.0`。

6.6 Sprint 4（第 8 周）：稳定化与发布

要点 | Sprint Goal

在不破坏任何质量门禁的前提下打磨体验、补齐可观测性，完成 v1.0.0 发布与最终答辩；按剩余容量完成 PO 选定的 Identity 扩展。

6.6.1 Sprint Backlog（PO 按价值从上到下选择，必选项加粗）

任务	内容	PBI	模块 / 层	负责	工时
T4-01	**PB-46** develop → main 发布 PR、打 `v1.0.0` 标签、编写 GitHub Release Notes（新功能、已知问题、部署步骤）；验证 `docker compose up -d --build` 一键部署	PB-46	CI / 部署	E	3

任务	内容	PBI	模块 / 层	负责	工时
T4-02	**PB-40** 管理员禁用账号：`AdminUserService.disable` + `POST /api/v1/admin/users/{id}/disable` + 删除该用户全部 Redis 会话（Spring Session 按主体名索引查找）+ 领域/应用测试 + 审计	PB-40	identity 全层	A、B	8
T4-03	**PB-42** 前端：分页/骨架屏/错误重试、Markdown 实时预览与代码高亮	PB-42	frontend	D	6
T4-04	**PB-44** Grafana 面板：`co_outbox_backlog`、`co_outbox_dead_letters`、`co_reputation_mismatches`、接口延迟	PB-44	deploy	E	3
T4-05	PB-47 管理页：待认证学生列表与认证、创建教职工账号	PB-47	frontend	C	4
T4-06	PB-41 / PB-43 / PB-45（容量允许时）	—	—	—	—
T4-07	更新 README（运行、部署、账号）、补充 ADR（如 ADR-010 禁用账号会话清理）、整理用户手册	—	文档	C	3
T4-08	最终演示彩排（≤ 15 分钟）与项目总回顾	—	全体	全员	1×5

6.6.2 最终答辩演示脚本（建议 15 分钟）

1. （1 分钟）产品愿景与团队分工；展示 GitHub Insights → Contributors。
2. （2 分钟）架构：上下文映射图、模块结构、一条 ADR 的取舍；现场运行 `ArchitectureTest`。
3. （6 分钟）用 demo 账号走完：提问 → 回答 → 投票 → 教师认证 → 发起悬赏 → 采纳 → 声誉与徽章变化 → 排行榜 → 搜索命中“已采纳” → 通知与 @提醒。
4. （2 分钟）质量：CI 流水线历史、覆盖率报告、端到端测试；演示一次事件重放的幂等性。
5. （2 分钟）过程：燃尽图、速度变化、一次典型的 PR 评审讨论、回顾会改进项的落实。
6. （2 分钟）问答。

第 7 章 Git / GitHub 团队协作规范

7.1 协作模式：PO 个人仓库 + 协作者

本项目**不创建 GitHub Organization，也不设仓库管理员。**团队的产品经理（PO）在自己电脑的 IntelliJ 中导入起始代码，推送到 **PO 个人账号下的 GitHub 仓库**；其他成员作为协作者（Collaborator）从该仓库克隆，把各自的功能分支推送回同一仓库，再通过 Pull Request 合并。这就是"共享仓库模型"，不使用 Fork。

参与者	在仓库中的身份	可以做 / 应该做
PO（产品经理）	仓库所有者（Owner）	创建仓库并推送初始代码；邀请协作者；配置默认分支、合并选项、分支保护和 Projects 看板。**与其他成员一样遵守 PR 规则**，不得利用所有者权限直接推送或绕过评审
其他成员	协作者（写权限）	克隆仓库；推送 `feature/*` 等分支；创建与评审 PR；管理 Issue、标签、Milestone；Sprint 末创建发布 PR 和版本标签
任课教师	协作者（或公开仓库的访问者）	查看提交历史、PR、Issue、Actions、Insights，在评审会上核查（见 9.1 节）

选择共享仓库而不是 Fork 的原因：所有分支和 PR 都在一个仓库里，Insights 中的贡献统计完整；同仓库分支发起的 PR 会直接触发 CI；Issue、PR 与 Projects 看板可以自动关联。

> **注意 | 私有仓库与分支保护**
>
> GitHub 免费个人账号的**私有**仓库不能启用分支保护规则（公开仓库可以）；私有仓库需要 GitHub Pro 等付费计划。
>
> 建议 PO 先通过 GitHub Education 申请学生权益（包含 GitHub Pro），再创建**私有**仓库。若暂时无法获得，须经教师同意后使用公开仓库，并注意防止其他小组抄袭。

7.1.1 PO：在 IntelliJ 中导入起始代码并推送到个人仓库

1. 解压 `CampusOverflow-starter.zip`，把目录 `campus-overflow-student/` 重命名为 `campus-overflow`。确认根目录下有 `.gitignore`（已忽略 `target/`、`node_modules/`、`dist/`、`.idea/`、`*.iml`、`.env`）。
2. IntelliJ：File → Open，选择**项目根目录** `campus-overflow`（不是 `backend`）。在 `backend/pom.xml` 上右键 → Add as Maven Project；在 File → Project Structure 中把 SDK 设为 JDK 21；等待 Maven 依赖下载完成。

3. 在 Settings → Version Control → GitHub 中登录 PO 的 GitHub 账号。在终端执行一次 `git config --global init.defaultBranch main`，保证初始分支名为 `main`。
4. 菜单 Git → GitHub → Share Project on GitHub（旧版 UI 为 VCS → Share Project on GitHub）：仓库名填 `campus-overflow`，勾选 Private（见上方提示），Remote 保持 `origin`。在弹出的首次提交对话框中确认**没有** `.idea/`、`target/`、`node_modules/`，提交信息填 `chore: import starter code`，然后提交并推送。
5. 点击 IntelliJ 右下角（新版 UI 在左上角）的分支菜单 → New Branch，新建 `develop` 并 Push；在 GitHub 仓库的 Settings → General 中把 Default branch 改为 `develop`。
6. Settings → Collaborators → Add people：按 GitHub 用户名邀请全部成员和任课教师；被邀请者需在邮件或 github.com 的通知中点击接受。
7. 按 7.1.3 节配置合并选项与分支保护，**必须勾选“Do not allow bypassing the above settings”**，让 PO 本人同样受规则约束。
8. 在 PO 账号下新建 Projects 看板（Board/Table），在看板的 Settings → Manage access 中把成员加为 Write，再在仓库的 Projects 页签中关联该看板。
9. 在团队群里发布仓库地址 `https://github.com/<po-username>/campus-overflow`。此后 PO 和其他成员一样，只通过功能分支 + PR 提交代码。

代码清单 7-1 PO 初始化仓库（命令行等价做法）

```
# 与 IntelliJ 操作等价的命令行做法（PO 执行一次即可）
# 先在 GitHub 网页新建空仓库 campus-overflow（不要勾选 README / .gitignore / License）
cd campus-overflow
git init -b main
git add . && git status            # 确认没有 target/、node_modules/、.idea/
git commit -m "chore: import starter code"
git remote add origin git@github.com:<po-username>/campus-overflow.git
git push -u origin main
git switch -c develop && git push -u origin develop
```

7.1.2 团队成员：接受邀请并从 PO 仓库克隆

1. 接受协作邀请后，在 IntelliJ 欢迎页选择 Get from VCS（或 File → New → Project from Version Control），URL 填 `git@github.com:<po-username>/campus-overflow.git`（或 HTTPS 地址），克隆到本地。
2. 打开项目后同样在 `backend/pom.xml` 上 Add as Maven Project，SDK 设为 JDK 21。
3. 配置提交身份：`git config --global user.name "<姓名拼音>"`、`git config --global user.email "<GitHub 账号绑定的邮箱>"`。**邮箱不一致时，你的提交不会计入 Insights → Contributors 的统计**，会直接影响个人贡献考核。

4. 配置 SSH Key（`ssh-keygen -t ed25519` 后把公钥添加到 GitHub），或在 IntelliJ 中登录自己的 GitHub 账号后使用 HTTPS。
5. 切换到 `develop`，从它创建功能分支（见 7.2 节）；以后只把自己的功能分支推送到 `origin`，**不要直接推送 develop 或 main 分支**（分支保护会拒绝）。
6. 完成 Sprint 0 的破冰 PR（T0-10），验证“克隆 → 分支 → 推送 → PR → 评审 → 合并”全流程。

7.1.3 仓库设置与分支保护（由 PO 在 Settings 中配置）

- Settings → General：Default branch = `develop`；勾选 “Automatically delete head branches”；合并方式只保留 “Allow squash merging”（feature→develop）和 “Allow merge commits”（develop→main）。
- Settings → Branches → Add branch protection rule（或 Rules → Rulesets）：分别为 `main` 和 `develop` 配置下表规则。使用 Rulesets 时，**Bypass list 保持为空**。
- 首次运行 CI 后，“必须通过的状态检查”里才能搜到 Job 名称。所以 PO 应在 T0-04 的 CI 改造合并后，再回来勾选这些检查。

规则	develop	main
必须通过 Pull Request 合并	是	是
所需 Approve 数	≥ 1（非作者）	≥ 2（含 SM 或 PO）
代码所有者评审（CODEOWNERS）	建议开启	开启
新提交后撤销已有批准	是	是
必须通过的状态检查	后端构建与验证、前端构建	同左 + 容器镜像构建
合并前分支必须是最新的	是	是
必须解决所有评审对话	是	是
禁止强制推送 / 禁止删除	是	是
不允许绕过上述规则（含仓库所有者 PO）	是	是

7.2 分支策略

采用与 CI 触发条件（`main`、`develop`、`sprint/**`）匹配的简化 Git Flow：

分支	来源 → 去向	命名示例	说明
`main`	— ← `develop`/`hotfix/*`	`main`	只存放可发布版本；每个 Sprint 末通过发布 PR 合入并打标签

分支	来源 → 去向	命名示例	说明
develop	— ← feature/*	develop	日常集成分支，始终保持 CI 绿色
feature/*	develop → develop	feature/23-question-accept	一个 Task/Story 一个分支（<Issue 号>-<英文短描述>），生命周期 ≤ 3 天
fix/*	develop → develop	fix/57-vote-delta	非紧急缺陷
sprint/*（可选）	develop → develop	sprint/2-integration	跨多人的集成联调分支，会触发 CI
hotfix/*	main → main + develop	hotfix/v0.3.1-login	已发布版本的紧急修复

代码清单 7-2 一个功能分支的完整生命周期

```
# origin 即 PO 的个人仓库；所有成员都向它推送自己的功能分支
git switch develop && git pull --rebase
git switch -c feature/23-question-accept
# …红→绿→重构，小步提交…
git add -p && git commit -m "feat(qa): 实现 Question.accept 采纳规则 (#23)"
git fetch origin && git rebase origin/develop          # 推送前同步，冲突在本地解决
git push -u origin feature/23-question-accept          # 然后在 GitHub 上创建 PR 到 develop
```

7.3 提交信息规范（Conventional Commits）

```
<type>(<scope>): <中文简述，≤ 50 字> (#<Issue 号>)

[可选正文：为什么这么改、有何取舍]
[可选页脚：Closes #23 / BREAKING CHANGE: …]

type ∈ feat | fix | test | refactor | docs | chore | ci | perf | style
scope ∈ identity | qa | reputation | discovery | shared | bootstrap | frontend | ci | docs
```

- 好的例子：test(reputation): 覆盖赞改踩 -12 的声誉增量 (#31)、fix(qa): 已删除回答不能被采纳 (#40)。
- 坏的例子：update、修改了一些东西、final version 2、一次提交 2000 行跨三个上下文。
- 粒度：一次提交只做一件事并能通过编译；建议“写测试 / 让测试通过 / 重构”分别提交，便于评审时看到 TDD 过程。
- 禁止提交：target/、node_modules/、.idea/、.env、真实密钥（已在 .gitignore 中）。

7.4 Issue 与 Projects 看板

标签组	取值	用途
type:	story / task / bug / tech-debt / spike / docs	区分 PBI、任务、缺陷与技术债

标签组	取值	用途
ctx:	identity / qa / reputation / discovery / frontend / infra	对应限界上下文或方向，便于统计与路由评审人
priority:	must / should / could / wont	MoSCoW 优先级
status:（可选）	blocked / needs-discussion	暴露阻碍，站会重点讨论

- Milestone = Sprint（`Sprint 0` … `Sprint 4`），描述中写 Sprint Goal 与起止日期。
- 看板由 PO 在个人账号下创建并关联到仓库，成员在 Manage access 中获得 Write 权限；看板列：`Backlog` → `Sprint Backlog` → `In Progress` → `In Review` → `Done`；自定义字段：Story Points、Sprint、Rank、Owner。
- Story Issue 下用 Task List 列出子任务，或建立子 Issue 并在正文中用 `- [ ] #45` 关联。
- PR 描述中写 `Closes #23`，合并后 Issue 自动关闭并移入 Done；SM 每次站会检查看板与实际一致。
- 缺陷必须使用 Bug 模板（复现步骤、期望结果、实际结果、错误码/日志），并关联引入它的 PR（若可追溯）。

代码清单 7-3 批量导入 Issue 示例（由 PO 执行）

```
# 用 GitHub CLI 批量导入 Backlog（backlog.csv：id,title,labels,points）
while IFS=, read -r id title labels points; do
  gh issue create --title "[$id] $title" --label "type:story,$labels" \
      --milestone "Sprint 1" --body-file .github/ISSUE_TEMPLATE/story-body.md
done < backlog.csv
```

7.5 Pull Request 与 Code Review

7.5.1 PR 规则

- 一个 PR 只对应一个 Task/Story；建议变更 ≤ 400 行（不含自动生成文件）；超过时拆分。
- 标题同提交规范；描述使用附录 B 的 PR 模板：关联 Issue、改动说明、测试证据（命令输出或截图）、自检清单。
- 创建 PR 前本地执行对应模块的 `mvn test` 与 `ArchitectureTest`；CI 红灯的 PR 不请求评审。
- 评审人：所改上下文的主责人（由 CODEOWNERS 自动指派）+ 任意一名其他成员；24 小时内给出首次评审。
- 作者负责回复每条评论并点击 Resolve；有分歧时在 PR 中讨论，必要时升级到站会或写 ADR。

代码清单 7-4 CODEOWNERS 示例（@member-b 为 PO）

```
# .github/CODEOWNERS（示例：按上下文指派第一评审人）
/backend/co-qa/          @member-a @member-b
/backend/co-reputation/  @member-b @member-c
/backend/co-discovery/   @member-c @member-d
/backend/co-identity/    @member-a
/backend/co-shared-kernel/ @member-e @member-a      # 共享内核变更需两人评审
/frontend/               @member-d
/.github/  /deploy/      @member-e @member-b
```

7.5.2 Code Review 检查清单

类别	检查项
DDD 与架构	业务规则是否在聚合/值对象/策略中，而不是写在应用服务的 if-else 里？领域层是否零框架依赖？跨上下文是否只经 api 或事件？计数器是否走原子更新？
正确性	是否满足验收标准与测试？边界值（10/150 字符、5/6 个标签、10/500 积分、1/14 天）是否覆盖？异常类型与错误码是否符合 3.7 节？
事务与事件	写操作是否在 `@Transactional` 中？事件是否在保存之后、同一事务内发布？事件处理器是否幂等（重复投递安全）？
测试	是否先有失败测试再有实现？新增规则是否有单元测试？**是否改动了已有断言或删除了测试？**（一票否决）
可读性	命名是否符合统一语言？方法是否短小？是否有无用代码、调试输出、被注释掉的大段代码？
安全	是否在接口层泄露个人信息（学号、邮箱）？权限判断是否只依赖前端？Markdown 是否经净化后输出？

> **提示｜评审意见的写法**
>
> 用“问题 + 依据 + 建议”格式，例如：“这里在应用服务里判断 `answerCount > 0`，属于领域规则（ADR-008/3.6 节），建议移到 `Question.softDelete`，并补一条测试。”避免只写“LGTM”。

7.6 IntelliJ IDEA 配置要点

项目	配置
导入工程	PO：File → Open 打开解压后的项目根目录；成员：Get from VCS 从 PO 仓库克隆。两者都在 `backend/pom.xml` 上 Add as Maven Project，SDK 选 JDK 21，Maven 使用 Bundled 或本地 3.9+（详见 7.1.1/7.1.2）
编码与格式	Settings → Editor → File Encodings 全部 UTF-8；统一代码风格（可导入 Google Java Style 或团队约定的 `.editorconfig` 并提交）
定位任务	Settings → Editor → TODO 新增模式 `\bTODO\(S[0-9]\)`，在 TODO 工具窗口按 Sprint 过滤待实现方法

项目	配置
运行测试	在测试类或方法上点击运行图标；“Run with Coverage” 查看领域层覆盖率；为每个上下文保存一个 JUnit 运行配置
启动应用	Spring Boot 运行配置：主类 `CampusOverflowApplication`，Active profiles = `dev`（Sprint 3 后可用 `dev,demo`）
接口调试	HTTP Client（`.http` 文件）：先请求 `/api/v1/auth/csrf`，后续请求带 `X-XSRF-TOKEN: {{xsrf}}` 头，会话 Cookie 自动保存
Git 集成	使用 Git 工具窗口查看分支图、交互式暂存（Commit 面板勾选行）、解决冲突（三方合并视图）；安装 GitHub 插件可在 IDE 内评审 PR
数据库	Database 工具窗口连接 `localhost:3306/campus_overflow`（campus / campus_dev_pwd），查看 `outbox_event`、`rep_ledger` 等表

第 8 章 CI/CD 与质量门禁

8.1 流水线结构

`.github/workflows/ci.yml` 在推送 `main`/`develop`/`sprint/**` 及向 `main`/`develop` 发起 PR 时触发：

Job	阶段	命令	验证内容
backend	1 编译	`mvn -B -f backend/pom.xml clean compile`	27 个模块可编译、Maven 依赖无环
backend	2 单元测试	`mvn … test`	领域测试 + 应用层用例测试（毫秒级、无外部依赖）
backend	3 架构约束	`mvn … test -pl co-bootstrap -Dtest=ArchitectureTest`	FF-1 ~ FF-6
backend	4 覆盖率门禁	`mvn … verify -DskipITs -Djacoco.skip=false`	各领域模块行覆盖率 ≥ `domain.coverage.minimum`
backend	5 集成测试	`mvn … verify -Dsurefire.skip=true`	Testcontainers 拉起 MySQL + Redis 的端到端链路
frontend	类型检查 / 构建	`npm run typecheck` / `npm run build`	TypeScript 类型正确、可构建
images	镜像构建（仅 push）	`docker build -f deploy/*.Dockerfile`	后端与前端镜像可构建（持续交付的前提）

8.2 为什么需要“渐进式质量门禁”

起始代码的测试在实现完成前必然失败。如果原样使用流水线，`develop` 会连续数周红灯，分支保护的“必须通过状态检查”将阻止所有合并；而如果为此关闭检查，又失去了 CI 的意义。解决办法是：**按 Sprint 范围逐步扩大必须通过的测试集合**，并把这个决定写成 ADR-009。

代码清单 8-1 ci.yml 渐进式门禁改造示例（片段）

```
env:
  # 每个 Sprint 计划会后由 E 同学更新（逗号分隔的模块路径）
  # S0: co-shared-kernel,co-identity/co-identity-domain
  # S1: 追加 co-qa/co-qa-domain,co-qa/co-qa-application
  # S2: 追加 co-reputation/co-reputation-domain,co-reputation/co-reputation-application
  # S3: 删除本变量，恢复全量 mvn test / verify
  SPRINT_MODULES: co-shared-kernel,co-identity/co-identity-domain,co-qa/co-qa-domain,co-qa/co-qa-application

      - name: 单元测试（Sprint 范围）
        run: mvn -B -f backend/pom.xml test -pl "$SPRINT_MODULES" -am
      - name: 覆盖率门禁（Sprint 范围）
        run: mvn -B -f backend/pom.xml verify -DskipITs -Djacoco.skip=false -pl "$SPRINT_MODULES" -am
      - name: 集成测试
        run: mvn -B -f backend/pom.xml verify -Dsurefire.skip=true
        continue-on-error: true      # S3 移除此行，集成测试成为硬门禁
```

注意 | 红线

渐进门禁只能"缩小运行范围"，不能通过 `-Dmaven.test.failure.ignore=true`、`@Disabled`、删除测试或修改断言来"变绿"。CI 配置的每次修改都必须经 PR 评审，教师会检查 `ci.yml` 的提交历史。

8.3 各 Sprint 的质量门槛

门禁	S0	S1	S2	S3	S4
编译 + ArchUnit	必须绿	必须绿	必须绿	必须绿	必须绿
单元测试范围	共享内核、Identity	+ Q&A	+ Reputation	全部模块	全部模块
领域覆盖率	—	co-qa-domain ≥ 70%	+ co-reputation-domain ≥ 70%	根 POM 设为 0.70	≥ 70%（挑战 80%）
应用层测试	—	Q&A 已有 + 新增	+ Reputation 自建	+ Discovery 自建	全部
集成测试	允许失败	允许失败	允许失败	必须绿	必须绿
前端构建	必须绿	必须绿	必须绿	必须绿	必须绿

按模块设置覆盖率门槛：在对应领域模块的 `pom.xml` 中覆盖父 POM 属性即可，例如在 `co-qa/co-qa-domain/pom.xml` 中加入 `<properties><domain.coverage.minimum>0.70</domain.coverage.minimum></properties>`；Sprint 3 再统一修改根 POM。

8.4 持续交付

- 每次推送到 `main`/`develop` 时 `images` Job 构建镜像（标签为 commit SHA），证明"随时可部署"。

- 扩展（推荐 S4 完成）：在 `main` 打 `v*` 标签时把镜像推送到 GitHub Container Registry（`ghcr.io/<po-username>/campus-overflow-backend:v1.0.0`），并自动创建 GitHub Release。
- 部署：`docker compose -f deploy/docker-compose.yml up -d --build`（含 `--profile observability` 时启动 Prometheus 与 Grafana）；生产 profile 必须通过环境变量注入 `CO_FIELD_KEY`。

8.5 CI 红灯处理流程

1. `develop` 变红后，提交者在 30 分钟内于群内认领，SM 在看板上建 `type:bug` 的 P0 Issue。
2. 能快速修复则提交修复 PR；不能则 `git revert` 引起问题的合并提交（同样走 PR），先恢复绿色再排查。
3. 红灯期间除修复 PR 外暂停向 `develop` 合并；回顾会统计红灯次数与恢复时长（MTTR）。

第 9 章 Sprint 验收与考核评价

9.1 Sprint 评审会的验收内容

每个 Sprint 的验收由“可运行的软件”与“GitHub 协作证据”两部分组成，教师在评审会上逐项核查：

维度	核查方式	合格标准
可运行的软件	从 `main` 的版本标签现场启动，按 Sprint 演示脚本操作；必要时用 Swagger 调用接口	Sprint Goal 达成；纳入的 Must 级 PBI 全部满足 DoD；无阻断性缺陷
代码提交历史	`git log --graph --oneline`、提交信息、提交时间分布	提交信息符合规范；每人每周 ≥ 3 次有意义提交；没有“最后一晚一次性提交”
贡献者维度	Insights → Contributors / Pulse；按作者统计 PR、评审、关闭的 Issue	每人都有领域层与应用层 PR；任何成员的有效贡献不低于团队平均的 50%
Pull Request 与 Code Review	抽查 3 ~ 5 个已合并 PR	全部经 PR 合入；有非作者 Approve；评审意见有实质内容并被处理；PR 关联 Issue
Issue 使用	Projects 看板、Milestone 完成率	每个 PBI 有 Issue 与验收标准；看板状态与实际一致；缺陷用模板登记
分支策略	分支列表与网络图；仓库 Settings 中的保护规则（PO 现场展示）	全体成员在 PO 仓库中以协作者身份推送功能分支；分支命名规范、合并后删除；`main` 只接受发布 PR；无强制推送；保护规则开启“不允许绕过”
CI/CD 状态	Actions 页面	`develop` 与 `main` 最新提交为绿色；红灯恢复时间 < 24 小时；Sprint 末有版本标签与 Release
Scrum 工件	Milestone 描述、燃尽图、回顾记录 `docs/scrum/sprint-N.md`	Sprint Goal 清晰；燃尽图可解释；回顾改进项被落实或说明原因

9.2 评分方案

考核项	权重	评分依据
Sprint 评审（S0 ~ S3 各 10%）	40%	每次按 9.1 节核查表打分：软件 50%、协作证据 30%、Scrum 工件 20%
最终产品与答辩（S4）	25%	功能完整度、端到端演示、架构讲解、问答表现
工程质量	20%	全部测试/ArchUnit/覆盖率/集成测试通过情况，代码可读性，ADR 与文档质量
过程与协作	15%	PR/评审/Issue 的规范性与持续性，CI 健康度，回顾改进的落实

个人成绩 = 团队成绩 × 个人贡献系数（0.6 ~ 1.2）。贡献系数综合以下三方面确定：

- GitHub 数据（约 50%）：合并的 PR 数量与质量、领域/应用层代码占比、完成的评审次数与质量、关闭的 Issue；**不以代码行数作为唯一依据。**
- 组内互评（约 30%）：每个 Sprint 回顾时匿名互评（责任心、沟通、技术贡献）。
- 教师观察（约 20%）：评审会与答辩中的讲解与回答，能否解释自己写的代码。

9.3 学术诚信与红线

红线	后果
修改已有测试的断言、删除或 `@Disabled` 已有测试以求“变绿”	涉及的 PBI 判为未完成；情节严重者团队过程分清零
向 `main`/`develop` 强制推送或绕过 PR 直接提交（包括 PO 利用仓库所有者权限关闭保护或绕过评审）	该 Sprint 协作证据项不得分
冒用他人提交（修改 author）、代他人提交以“刷贡献”	相关成员贡献系数取下限
提交密钥、个人隐私数据（真实学号、手机号）到仓库	立即清理历史并扣分
直接复制其他小组代码	按学校学术不端规定处理

> **提示｜关于 AI 编程助手**
>
> 允许使用 AI 工具辅助理解代码与生成测试草稿，但须在 PR 描述中注明使用情况，并保证本人能够解释每一行提交的代码；评审会上教师会随机请成员讲解自己 PR 中的实现。

9.4 最终交付物清单

交付物	位置	截止
源代码与完整提交历史	GitHub 仓库 `main` 分支，标签 `v1.0.0`	第 8 周评审会前
GitHub Release（含 Release Notes 与部署说明）	Releases 页面	第 8 周
DDD 设计文档（上下文映射、统一语言、事件目录、领域模型）	`docs/ddd/`	S0 初版，S3 定稿
ADR（含新增的 009 及以后）	`docs/adr/`	随决策提交
Scrum 记录（每个 Sprint 的计划、评审、回顾、燃尽图）	`docs/scrum/sprint-0.md` … `sprint-4.md`	每个 Sprint 末
测试报告与覆盖率报告截图	Actions 构件 / `docs/reports/`	第 8 周
答辩演示文稿与演示视频（≤ 5 分钟）	`docs/final/`	第 8 周

交付物	位置	截止
个人总结（贡献、收获、改进）	提交至教学平台	第 8 周

附录

附录 A 107 个 TODO 方法与 PBI / Sprint 映射

原 README 中的标记 S2 对应本指导书 Sprint 1，标记 S3 对应 Sprint 2（Reputation）与 Sprint 3（Discovery）。“自建”表示没有现成测试，需团队按 DoD 自行编写应用层单元测试。

类	层	待实现方法	数	PBI	Sprint	规格来源
Answer	领域	submit、edit、endorse、softDelete、ensureNotDeleted	5	PB-09/12/13/14	S1	AnswerTest、QuestionTest
Comment	领域	post	1	PB-15	S1	VoteAndCommentTest
MarkdownBody	领域	ofQuestion、ofAnswer	2	PB-06/09	S1	QuestionTest
Question	领域	post、edit、accept、softDelete、ensureOpenForAnswers、ensureNotDeleted	6	PB-06/10/12/13	S1	QuestionTest
Tags	领域	names	1	PB-06/16	S1	QuestionTest
Vote	领域	cast、change、checkValue	3	PB-11	S1	VoteAndCommentTest
QaQueryApiImpl	应用	findQuestion	1	PB-17	S1	自建
QuestionQueryService	应用	list、detail、popularTags	3	PB-07/08/16	S1	自建
AnswerCommandService	应用	submit、edit、delete、accept、endorse、loadQuestion、loadAnswer	7	PB-09/10/12/13/14	S1	AnswerAndVoteServiceTest
CommentService	应用	post	1	PB-15	S1	自建
QuestionCommandService	应用	post、edit、delete、load	4	PB-06/12/13	S1	自建、集成测试
VoteService	应用	cast、resolve	2	PB-11	S1	AnswerAndVoteServiceTest
AcademicTerm	领域	termKey、weekKey、monthKey	3	PB-26	S2	BountyAndAccountTest
BadgePolicy	领域	forAcceptedCount、forReputation	2	PB-23	S2	BountyAndAccountTest
Bounty	领域	open、award、close、isDue、ensureOpen	5	PB-24/25	S2	BountyAndAccountTest
ReputationAccount	领域	open、applyReputation、freezePoints、spendFrozen、refundFrozen、ensureFrozen、requirePositive	7	PB-20/21/24/25	S2	BountyAndAccountTest
ReputationRulebook	领域	voteEffect、voteDelta、applyDailyCap、initialPoints	4	PB-20/22	S2	ReputationRulebookTest
BadgeService	应用	grant	1	PB-23	S2	自建
BountyService	应用	open、settleOnAccept、expireDue、latest、closeAndRefund	5	PB-24/25	S2	自建
LeaderboardService	应用	on、top	2	PB-26	S2	自建
ReconciliationService	应用	reconcile	1	PB-28	S2	自建
ReputationApiImpl	应用	summary、reputations	2	PB-27	S2	自建

类	层	待实现方法	数	PBI	Sprint	规格来源
ReputationEventHandlers	应用	on×5（UserRegistered、AnswerAccepted、VoteCast、AnswerSubmitted、AnswerEndorsed）	5	PB-20～23	S2	自建、集成测试
ReputationQueryService	应用	profile、ledger	2	PB-27	S2	自建、集成测试
MentionParser	领域	extract	1	PB-35	S3	DiscoveryDomainTest
Notification	领域	create、markRead	2	PB-34/36	S3	DiscoveryDomainTest
RelatedQuestionPolicy	领域	jaccard、rank	2	PB-33	S3	DiscoveryDomainTest
SearchQuery	领域	terms()、terms(String)	2	PB-31	S3	DiscoveryDomainTest
DiscoveryQueryService	应用	search、related	2	PB-31/33	S3	自建、集成测试
NotificationProjector	应用	on×6、notify、name	8	PB-34/35	S3	自建、集成测试
NotificationService	应用	list、unreadCount、markRead、markAllRead、toView	5	PB-36	S3	自建、集成测试
SearchIndexProjector	应用	firstTime、on×9	10	PB-32	S3	自建、集成测试

合计：Sprint 1 = 36 个，Sprint 2 = 39 个，Sprint 3 = 32 个，共 107 个，分布在 32 个文件中。

附录 B 模板

B.1 User Story Issue 模板（.github/ISSUE_TEMPLATE/story.md）

```
---
name: 用户故事
about: Product Backlog Item
labels: type:story
---
## 用户故事
作为 <角色>，我希望 <功能>，以便 <价值>。

## 验收标准 (Given / When / Then)
- [ ] Given … When … Then …
- [ ] 错误场景：… 返回 <HTTP 状态> <错误码>

## 规格与实现落点
- 相关测试：`QuestionTest#askerCanAcceptAnAnswerOnce`
- 涉及方法：`Question.accept`、`AnswerCommandService.accept`
- 发布/订阅事件：`AnswerAcceptedEvent`

## 估算与任务
Story Points:    | Sprint:
- [ ] #任务 1  - [ ] #任务 2
```

B.2 Pull Request 模板（.github/pull_request_template.md）

```
## 关联 Issue
Closes #

## 改动说明
<!-- 做了什么、为什么这样做；涉及哪个上下文与哪一层 -->

## 测试证据
<!-- 粘贴 mvn test 输出摘要或截图；前端改动附截图/GIF -->

## 自检清单
- [ ] 本地 `mvn test -pl <模块> -am` 通过，`ArchitectureTest` 通过
- [ ] 业务规则位于领域层，领域层无框架依赖
- [ ] 新增规则有单元测试；**未修改任何已有测试断言**
- [ ] 异常类型与错误码符合 docs/错误码.md
- [ ] 写操作有事务；事件在保存后发布；消费端幂等
- [ ] 无调试代码、无密钥、无无关格式化改动
- [ ] 使用了 AI 辅助：否 / 是（说明：…）
```

B.3 Sprint 记录模板（docs/scrum/sprint-N.md）

```
# Sprint N（YYYY-MM-DD ~ YYYY-MM-DD）
## Sprint Goal
## 计划：纳入的 PBI 与故事点（承诺 xx 点 / 容量 xx 小时）
## 每日站会摘要（日期 | 阻碍 | 处理）
## 评审：演示结果、被接受/退回的 PBI、教师反馈
## 数据：完成点数（速度）、燃尽图截图、PR 平均周转时间、CI 红灯次数与恢复时长
## 回顾：做得好的 / 待改进的 / 改进行动（负责人、Issue 链接）
```

附录 C 常见问题与排错

现象	原因与处理
`UnsupportedOperationException: TODO(Sx)…`	该方法尚未实现，属正常现象；若出现在发件箱日志中，检查是否已按 6.3.4 节实施行走骨架
声誉不变化	声誉是异步结算的：先查 `outbox_event` 的 `published_at` 与 `last_error`，再查 `rep_ledger`
发件箱事件全部积压	某个监听器持续抛异常导致批内阻塞；修复后可执行 `UPDATE outbox_event SET attempts = 0, locked_until = NULL WHERE published_at IS NULL`
事件重复导致加分两次	消费端未幂等：账本依赖 `(event_id, user_id, reason)` 唯一键，投影使用 `processed_event` 表，通知使用 `(source_event_id, recipient_id)`
中文搜索没有结果	必须使用 compose 中的 MySQL（`ngram_token_size=2`、关闭停用词）；单个汉字关键词会被 DIS-1001 拒绝

现象	原因与处理
调用写接口返回 403 `AUTH-403-CSRF`	先 `GET /api/v1/auth/csrf`，之后请求头带 `X-XSRF-TOKEN`（值取自 `XSRF-TOKEN` Cookie）
登录返回 429 `AUTH-429`	1 分钟内失败 5 次触发限流；等待 1 分钟或清理 Redis 中的限流键
改了角色但权限没变	会话中缓存了 Actor，需要退出并重新登录
`mvn verify` 集成测试无法启动	本机 Docker 未运行或无法拉取镜像；可先单独 `docker pull mysql:8.0.39 redis:7-alpine`
ArchUnit 报错 FF-2	某上下文 import 了其他上下文的 domain/application 类；改为依赖对方 `*-api` 或订阅事件
合并冲突频繁	分支生命周期过长：任务拆到 ≤ 1 天，每天 `git rebase origin/develop`；避免多人同时修改同一文件的大段格式
乐观锁 409 `SYS-409-CONCURRENT`	并发修改同一聚合；前端提示刷新重试，属预期行为
推送时报 `Permission denied` / 403	尚未接受 PO 的协作邀请，或 SSH Key / HTTPS 凭据属于另一个账号；在 GitHub 通知中接受邀请并检查 `git remote -v` 指向 PO 仓库
推送 `develop` 被拒（protected branch）	属预期行为：只能推送自己的功能分支，再发起 PR
Insights 中看不到自己的贡献	本地 `user.email` 与 GitHub 账号邮箱不一致；在 GitHub Settings → Emails 中添加该邮箱，或修改本地配置后再提交
分支保护设置页提示需要升级	私有仓库在免费个人账号下不支持分支保护；PO 申请 GitHub 学生权益（Pro），或经教师同意改为公开仓库

附录 D REST 接口清单（摘自接口层代码）

上下文	方法与路径	说明
Identity	`GET /api/v1/auth/csrf`; `POST /auth/register`; `POST /auth/login`; `POST /auth/logout`; `GET /auth/me`	认证与会话
Identity	`GET /courses`; `GET /courses/mine`; `POST /courses`; `POST /courses/{id}/join`; `GET /courses/{id}/members`; `PUT /courses/{id}/members/{userId}`	课程与成员
Identity	`GET /admin/users`; `POST /admin/users`; `POST /admin/users/{id}/verify`; `PUT /admin/users/{id}/role`	管理员
Q&A	`GET/POST /questions`; `GET/PUT/DELETE /questions/{id}`; `GET /tags/popular`	问题与标签

上下文	方法与路径	说明
Q&A	`POST /questions/{id}/answers`；`PUT/DELETE /answers/{id}`；`PUT /questions/{id}/accepted-answer`；`POST /answers/{id}/endorsement`	回答、采纳、认证
Q&A	`PUT /votes`（targetType、targetId、value∈{−1,0,1}）；`POST /comments`	投票与评论
Reputation	`GET /users/{id}/reputation`；`GET /users/{id}/reputation/ledger`；`GET /leaderboard?period=`	声誉档案、流水、排行榜
Reputation	`GET/POST /questions/{id}/bounty`；`GET /admin/reputation/reconciliation`	悬赏与对账
Discovery	`GET /search/questions`；`GET /questions/{id}/related`	搜索与相关问题
Discovery	`GET /notifications`；`GET /notifications/unread-count`；`POST /notifications/{id}/read`；`POST /notifications/read-all`	通知中心

注：除第一行外，路径均省略前缀 `/api/v1`。完整的请求/响应结构以运行时 Swagger UI（`/swagger-ui.html`）为准。